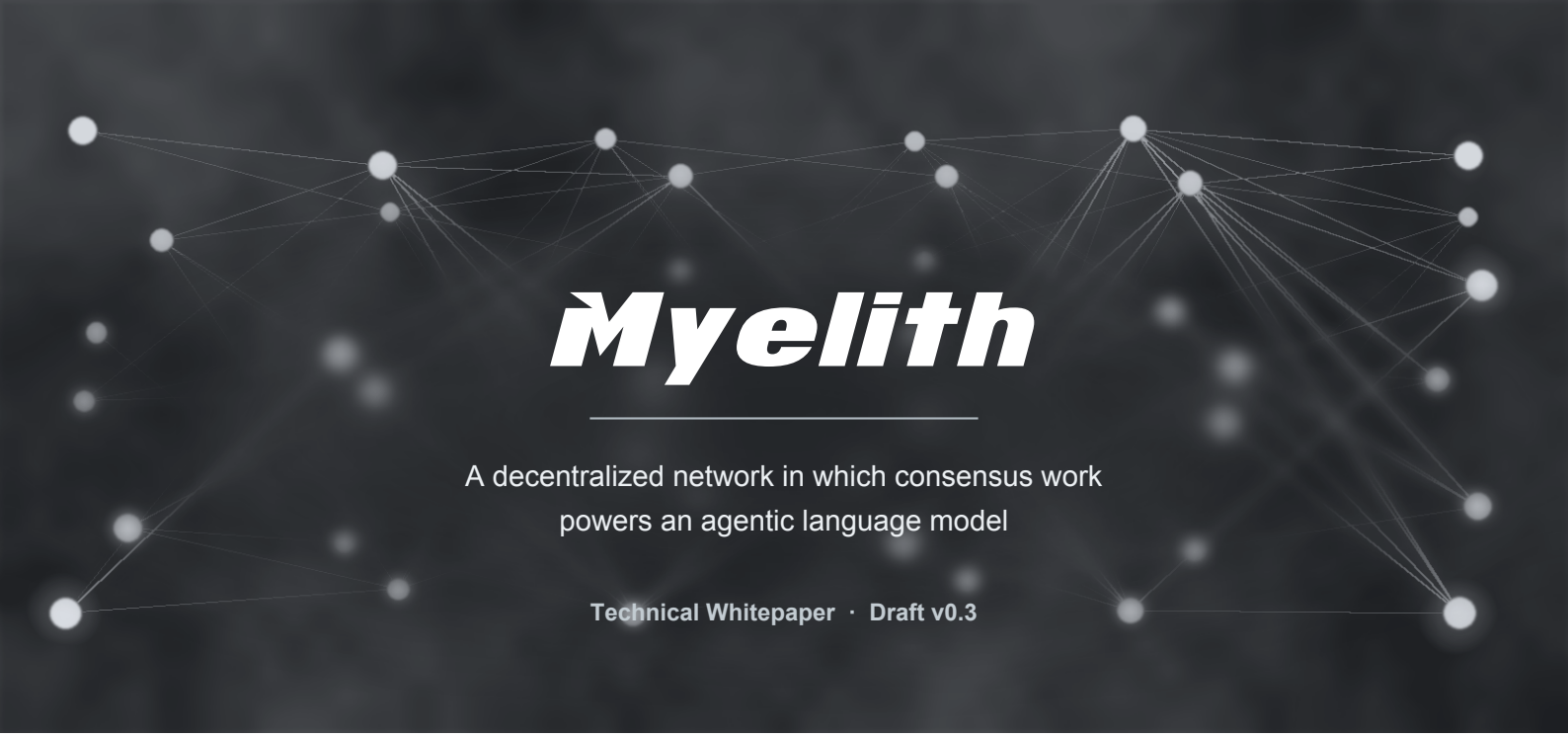

Myelith

A decentralized network in which consensus work powers an agentic language model

Technical Whitepaper · Draft v0.3

Joschka Benjamin Hänsler

August 2026

License: CC BY-SA 4.0 · No token exists · Comments welcome

Named Myelin through version 0.1; all versions are linked by the concept DOI.

Contents

Abstract

Proof-of-work blockchains purchase their security through the expenditure of compute and energy, yet the work performed is itself discarded entirely. Decentralized AI networks provide useful compute but secure no ledger. Myelith unifies both functions: miners jointly operate a large agentic language model (the *network model*) via pipeline parallelism, and the same cryptographically attested inference work ("Proof of Inference", PoI) determines compensation and feeds the voting weight of consensus. The native coin MYL closes the value cycle: users burn MYL for inference credits, and miners receive newly minted MYL in proportion to verified work (burn-and-mint equilibrium). We specify (i) a layered architecture that decouples consensus latency from inference latency, (ii) a verification model based on fully integer execution: since integer addition is associative, bit equality arises without any prescription over the order of operations, so that heterogeneous hardware participates without throughput loss and without competitive disadvantage; supplemented by injected control segments against the one-off targeted intervention and a selectable mode of confirmed delivery; we further document why the floating-point counter-designs do not hold, (iii) a token economy with a quantifiable security condition ($S_{min} = g/p^2$), (iv) a training procedure that uses free capacity, verifies data provenance rather than data content, and allows the network to grow its model incrementally, (v) an agent layer that makes the limit of verifiability at the boundary to the outside world visible and bounds damage through consensus-enforced session limits, (vi) an explicit confidentiality risk model with usage classes, and (vii) core data types and reference algorithms of an open-source implementation. Open points, the output quality of integer-quantized models at the target scale, the completeness of the execution specification, the indistinguishability of control segments, and the 50% redundancy overhead, are named as measurement questions with assigned milestones. Version 0.3 adds the measurement on a reference implementation: fully integer execution of a language model is bit-identically reproducible and costs 2.11 percent perplexity at 0.5 billion parameters and 1.14 percent at 7 billion, against the floating-point reference. The cost arises not in the representation of weights but in the intermediate stages of the compute path, and it sits only 0.30 percentage points above the independently measured floor of the quantization scheme itself. Also measured are a training step that leaves no floating-point state behind and a model enlargement that is exactly rather than approximately function-preserving.

1. Introduction

1.1 Two Wasted Resources

The Bitcoin network consumes energy on the scale of a mid-sized industrial nation to compute SHA-256 hashes whose sole purpose is to prove their own expenditure. Meanwhile, the compute for large language models is concentrated in the data centers of a few corporations; access, pricing, and model behavior are subject to centralized control. Both systems waste something. One wastes computational work, the other the possibility of open participation.

The obvious synthesis (mining work *is* AI work) has so far failed on three technical hurdles:

1. **Verification:** Classical PoW is asymmetric (hard to solve, trivial to check). LLM inference is symmetric: verification costs nearly as much as the computation itself.
2. **Latency:** A model that does not fit on a single machine must be distributed across the network; naive distribution fails on WAN latencies.
3. **Consensus coupling:** It is unclear how "I computed layers 40–60 correctly" becomes a block right without opening grinding and collusion attacks.

1.2 Contribution

This paper proposes an architecture that does not define these three hurdles away but addresses each with explicit costs: verification via redundancy plus sampling (cost: 50% overhead plus dispute windows), latency via pipeline parallelism with latency-aware pod formation (cost: seconds-range per pipeline pass, compensated by micro-batching and speculative decoding), and consensus coupling via the separation of fast BFT block production from epoch-wise PoI settlement (cost: compensation delayed by one dispute window).

Relative to v0.1, verification rests on a fundamentally new basis. v0.1 required bit-identical floating-point execution and therefore had to prescribe the order of all operations, which costs throughput and presupposes uniform rounding behavior across hardware, a condition the fast compute paths of modern accelerators do not meet. v0.2 instead executes inference entirely in integer arithmetic. Since integer addition is associative, bit equality then arises by itself, regardless of how the hardware parallelizes. Chapter 6.3 documents why the floating-point alternatives, fixed ordering, tolerance comparison, and software emulation, do not hold up under examination. Relative to v0.2, Chapter 6.9 records the findings of the reference implementation: bit width must be fixed per tensor, the execution specification must be obtained from the weights rather than the architecture description, and the gap to the floating-point reference decomposes measurably into a share owed to the quantization scheme and a share owed to the implementation. A further lesson concerns the verification model itself: a commitment must be formed over the computed quantities, not over the decision derived from them.

The claim is deliberately more modest than "better than centralized providers": The redundancy overhead forbids

efficiency leadership; Myelith therefore competes on censorship resistance, availability, open price formation, and the utilization of otherwise idle hardware.

1.3 Structure of This Paper

Chapter 2 delineates the design from related work. Chapters 3–4 specify the architecture and compute layer. Chapter 5 develops the token economy, Chapter 6 the formal verification model. Chapter 7 covers training and model development, Chapter 8 the agent layer, Chapter 9 confidentiality and the risk model, Chapter 10 governance and model provenance, Chapter 11 open research questions. Appendix A contains core data types and reference algorithms, Appendix B the incentive derivations.

2. Related Work and Differentiation

Qubic (uPoW / Aigarth) [2]. Qubic replaces hash puzzles with the training of neural networks; miners produce training solutions that simultaneously serve as proof of work for the Computor election. Qubic is the most direct predecessor of the consensus coupling in this work. Differences: Qubic verifies *training* contributions, whose utility is assessed statistically, while Myelith verifies *inference* segments objectively against a canonical reference; Qubic's work product is an evolving research system, Myelith's work product is an immediately usable service whose demand drives the coin cycle.

Bittensor (TAO) [3]. Bittensor is a marketplace of subnets in which validators qualitatively score miner responses (Yuma consensus). The scoring is subjective-statistical and has repeatedly shown gaming problems (weight copying, collusion). Myelith avoids subjective scoring entirely: a segment is correct or incorrect, decidable by hash equality and (in a dispute) by canonical recomputation (Ch. 6). There is no scope for judgment and no threshold an attacker could align with. The price: Myelith can operate only *one* canonical network model, not an open model market.

Petals [4]. Petals demonstrates BitTorrent-style pipeline serving of large open-weight models on volunteer hardware, the practical feasibility proof for our compute layer. However, Petals has neither verification nor incentives nor a ledger; its nodes are simply assumed to be honest. Myelith can be read as "Petals + verification + consensus + economy".

Gensyn / Verde and RepOps [5][15]. Gensyn verifies decentralized ML computation via *refereed delegation*: several providers compute the same task, and a dispute game decides as soon as they disagree, correct provided at least one is honest. The bisection game in Chapter 6.6 follows the same tradition (Truebit, Arbitrum). Central to this work is Gensyn's second contribution: RepOps, a library of reproducible operators that eliminates hardware non-determinism through fixed floating-point orderings, demonstrating bitwise reproducibility across different hardware. RepOps establishes that bitwise reproducibility across hardware boundaries is achievable and is thus the closest point of comparison to Chapter 6.2. Myelith nonetheless takes a different route: RepOps enforces reproducibility within floating-point arithmetic and pays for it with restricted parallelization; moreover, the library expressly covers only the single-device case, while reproducibility across multiple nodes with pipeline or tensor parallelism is named as future work, precisely the case that obtains in Myelith. Integer execution avoids both, since it requires no ordering prescription.

TOPLOC [14]. TOPLOC proposes locality-sensitive commitments over intermediate activations that reliably detect interventions on model, prompt, or precision while remaining robust to different GPU types and algebraic reorderings. With very compact proof sizes and validation faster than the original generation. TOPLOC addresses the trust problem toward a *single* inference provider and knows neither consensus nor economy nor sharding. Myelith examined this path and rejected it: tolerance-based commitments do not hold across chained execution and are adaptively attackable (Ch. 6.3). They remain, however, the most promising candidate should integer execution prove too restrictive, and are carried in Chapter 10 as a research direction.

Early integer transformer inference. The line begins before the works drawn on in Chapter 6. Dyadic arithmetic was first developed for an integer-only pipeline in convolutional networks [42], but it is tailored to linear and piecewise-linear operations and does not apply to the non-linear operations in transformers. The first work aiming at full integer inference for transformers replaced the square root in normalization with an L1-norm equivalent [41]; I-BERT [18] followed with integer polynomial approximations for softmax, GELU, and layer normalization. Myelith adopts the result of this line, not the method: what matters here is solely that fully integer execution is possible, since determinism follows from it (Ch. 6.2).

VeriLLM. VeriLLM is a publicly verifiable decentralized inference framework built on a blockchain substrate and shares several design goals with this work: auditable correctness, low verification overhead, deterministic accountability, task-type indistinguishability, and compatibility with heterogeneous hardware [43]. It avoids a verification bottleneck through an isomorphic architecture in which inference and verification roles run on the same compute workers, raising utilization while enlarging the set of possible verifiers. The difference from Myelith lies in purpose: VeriLLM secures the correctness of inference but no ledger. The work performed there carries no consensus, and no value cycle arises in which the same work determines both compensation and voting weight. The isomorphic role assignment is nonetheless a serious alternative to redundancy with r = 2 and is carried as such in Chapter 11, point 18.

Software floating-point emulation. Besides the ordering prescription [15] and tolerance comparison [14], a third route to platform-independent identical results exists: emulating floating-point operations entirely in software rather than using the hardware's arithmetic units. Optimistic and voting-based schemes presuppose this route in order to obtain consistent results across platforms. It is bit-exact and imposes no hardware requirements, but slows inference considerably, since every single operation is assembled from integer instructions. Myelith does not pursue it: if integer arithmetic is used anyway,

it is more consistent to execute the model itself in integers than to reconstruct floating point on top of them. The difference is substantial — in the first case one model operation corresponds to one machine operation, in the second to a sequence of dozens.

Integer inference (I-BERT, I-ViT, I-LLM) [18][19][20]. These works pursue a different goal than Myelith: they quantize transformers entirely to integer arithmetic in order to reduce memory footprint, latency, and energy consumption, particularly for edge devices [18][19][20]. Determinism is not a design goal there but a by-product. Precisely this by-product is constitutive for Myelith: since integer addition is associative, inference so executed yields bit-identical results without any prescription over the order of operations (Ch. 6.2). Myelith contributes nothing to quantization technique here, but rather the observation that integer execution resolves the verification question of distributed inference at its root, together with the demonstration of the additional stipulations this requires (Appendix B.5).

Numerical behavior of matrix units [21]. Studies of the matrix multipliers designed for AI show that these presently do not conform to IEEE-754 behavior and differ in rounding behavior, accumulator width, and normalization points between architecture generations of the same vendor, yielding non-reproducible results at the level of the individual matrix instruction [21]. That work examines four generations of one vendor; that other vendors follow the same pattern is plausible but not its subject. This finding is why Myelith does not take the floating-point route: an ordering prescription presupposes uniform rounding behavior of the individual instruction, and this condition is not met for the fast compute paths of modern accelerators.

Ora (opML) and zkML systems (EZKL, Modulus) [6]. Optimistic and zero-knowledge verification of individual ML inferences are established; in both cases, however, they serve as oracles *for* existing chains. Myelith inverts the relationship: inference is not a guest on a chain but its working foundation.

HadAgent [1]. HadAgent coins the term Proof-of-Inference for a consensus in which nodes earn block rights through deterministic LLM inference, the most direct precursor to Chapter 3.5, both terminologically and conceptually. The central difference lies in model size and thus the entire verification mechanics: HadAgent verifies via full recomputation of a single forward pass by master nodes in a two-tier architecture, the model must fit entirely on one node. Myelith addresses models that no single node can hold: pipeline sharding across pods, verification via redundancy plus bisection (dispute resolution through a single shard forward instead of full recomputation), work-weighted validator election, and a burn-and-mint cycle coupling coin and inference access. A second difference concerns the *source* of determinism. HadAgent establishes it through a prescribed execution environment, a fixed framework version and a fixed numerical precision. That is the class of assurance Chapter 6.3 examines and rejects: it binds every node to the same software and presupposes uniform rounding behavior across hardware. Myelith instead derives bit equality from the associativity of integer addition and prescribes neither order nor kernel nor framework. HadAgent is an architecture contribution and deliberately leaves the execution technique open; its evaluation was carried out in a single-node environment without networking.

Proof of Quality (PoQ) and PolyLink [7][8]. PoQ replaces computation verification with output quality assessment via lightweight evaluation models; PolyLink combines VRF-elected validator committees with LLM-as-a-judge scoring. Both accept subjective assessment as the price of speed, Myelith stays with objective decidability (cf. the Bittensor differentiation).

Blockchain-based federated learning. A substantial line of research combines blockchains with federated learning to make the provenance and integrity of training contributions verifiable. Proposals include Merkle-based provenance of data points and updates with compact on-chain metadata [30], zero-knowledge proofs for local training steps including forward and backward passes [31], and consensus-based detection of poisoned contributions [32][33]. The data provenance of Chapter 7.3 stands in this line and claims no novelty for the method as such. The difference lies in context: the works cited secure federated learning among few, mostly known organizations, frequently on permissioned blockchains. Myelith operates with anonymous, economically motivated participants in an open network where the same infrastructure simultaneously performs inference and carries consensus.

Byzantine-robust aggregation. The methods on which Chapter 7.4 relies originate in this literature: Krum selects the update with the smallest sum of distances to its neighbors [34]; coordinate-wise median and trimmed mean replace the mean with robust order statistics [35]; Bulyan combines both approaches. Established there is also the property on which Chapter 7.4 rests: coordinate-wise methods tolerate up to half of contributions being Byzantine [35]. Myelith adopts the median unchanged; its own contribution is limited to the observation that in integer arithmetic it requires only comparisons and thus remains verifiable within the model of Chapter 6. A known limitation applies: under non-identically distributed data and under attacker majorities these methods lose their guarantee.

Defense against injected instructions. The agent layer (Ch. 8.3) builds on the dual-LLM pattern [39] and its elaboration in CaMeL [40], which architecturally separate control flow from data flow so that retrieved content cannot influence execution. Myelith claims no novelty here. Its contribution lies in the connection to consensus: since budget, recipient list, and time window reside in the session contract and thus outside the model context, enforcement is not performed by a software layer but by the chain itself. An injected text therefore cannot shift the limits even if separation within the model fails.

DeServe [9]. DeServe examines how inference of large models can be made cheaper through decentralization and is thus related to the compute layer of this work. Verification and consensus are not its subject; its contribution lies on the cost side.

Render, io.net, Akash (DePIN compute). These networks broker GPU capacity as a commodity; the work secures no consensus, and there is no canonical model. They are

marketplaces; Myelith is an organism: one model, one ledger, one cycle.

Summary differentiation: The term Proof of Inference and the basic idea of inference as consensus work are anticipated by HadAgent and are not claimed as novel here. However, no existing system unifies (a) a single large, pipeline-distributed agentic model, (b) objectively verified inference as a consensus-relevant proof of work whose determinism follows from the arithmetic itself rather than from an execution mandate, and (c) a burn-and-mint cycle in which the mined coin is the access right to the work product. This combination is the contribution of this work.

3. Architecture

3.1 Design Goals and Base Assumptions

The network pursues three simultaneous goals that classical blockchains and decentralized compute networks have so far solved separately:

1. **Ledger security:** A tamper-proof, decentralized transaction register.
2. **Useful compute:** The computational work expended for security operates a large agentic language model (henceforth: *the network model*) instead of burning hashes.
3. **Closed value cycle:** The currency created by mining is simultaneously the means of payment for inference on the network model (burn-and-mint economy).

Base assumptions:

- Participant hardware is heterogeneous (consumer GPUs to data-center clusters) and connected via ordinary internet links (latency 20–200 ms, bandwidth 50 Mbit/s – 10 Gbit/s).
- Participants are economically rational and potentially Byzantine (up to a share f < 1/3 of weighted votes).
- The network model is too large for individual nodes (target size: 100B–1T+ parameters) and must therefore be sharded across multiple nodes.

3.2 Layer Model

The architecture strictly separates consensus from compute but couples both via cryptographic proofs of work:

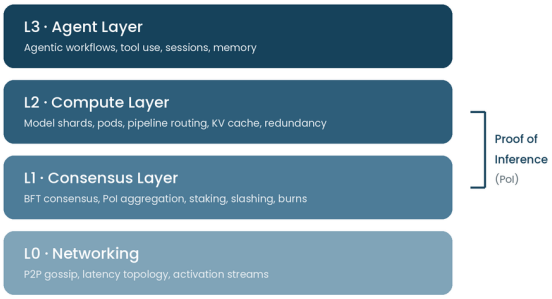

Figure 1: Layer model. Proof of Inference couples the compute and consensus layers.

Core decision: Consensus (L1) does *not* run on the inference results themselves, but on compact, verifiable *proofs of work* (Proof of Inference, PoI) produced by the compute layer. Block time thus remains independent of inference latency.

3.3 Network Roles

Role	Task	Hardware	Incentive
Shard miners	Each holds one model shard (contiguous layer group) in VRAM and computes forward passes	GPU ≥ 16–24 GB VRAM	Block reward proportional to attested inference work
Pod coordinator	Elected miner of a pod; orchestrates the pipeline, collects partial proofs, submits aggregated PoI	like shard miner + good connectivity	Coordination bonus
Validators	Run BFT consensus, spot-check PoI samples, manage stake/slashing	CPU-heavy, 1 GPU for spot checks	Share of fees + inflation reward
Checkers (fisherman)	Recompute randomly selected inference segments and report deviations	any GPU	Bounty from slashed stake
Gateways	Accept user requests, route to pods, stream results back	network-heavy	Share of the inference fee
Users	Burn coins for inference credits	–	Access to the network model

3.4 Verification at a Glance: Three-Tier Security Model

The system's most expensive problem is proving correct computation. The architecture combines three mechanisms with different cost–security profiles:

Tier 1, Deterministic redundancy (immediate, cheap):

Every inference segment is computed in parallel by r = 2 independently assigned pods. Since execution is fully integer and integer addition is associative, the results are bit-identical regardless of how each node parallelizes; plain commitment hashes are compared. The comparison is binary and parameter-free (Ch. 6.2). A redundancy factor of 2 costs 50% efficiency. That is the price of decentralization and enters the economics as an explicit line item.

Tier 2, Optimistic sampling (delayed, targeted):

Checkers fully recompute VRF-drawn segments (~1–3% of volume). On deviation, a **bisection game** starts (analogous to Truebit [10] and Arbitrum [11]): the dispute is narrowed binarily to the first divergent transition; only this single shard forward is recomputed on-chain by the validators. Since the correct result is canonical, attribution of fault is unambiguous. The loser is slashed; the checker receives a bounty.

Tier 3, zkML anchors (rare, expensive, maximally secure):

For particularly valuable results (e.g., completions of agentic transactions with financial effect), users can request a zero-knowledge proof of the inference for a surcharge. zk proofs for complete LLM forward passes are today still orders of magnitude too expensive for regular operation; the architecture treats them as an optional premium path and an upgrade path once zkML systems become efficient enough.

Determinism without an execution mandate: What is binding is the quantization scheme and a few arithmetic stipulations, not

the ordering, block partitioning, or kernel implementation (Ch. 6.2). Heterogeneous hardware thus participates without throughput loss and without competitive disadvantage.

Economic security: Shard miners deposit stake proportional to their reward capacity. The expected penalty (slash probability × stake) must exceed the expected gain from false computation; the sampling rate is the control lever and is adjusted via governance to the observed fraud rate.

3.5 Consensus Layer: Proof of Inference + BFT

3.5.1 Why Not Pure "Inference PoW"

Leader election directly via inference races (whoever computes first writes the block) would be manipulable (grinding over inputs) and would couple block time to inference latency. Instead:

3.5.2 Two Decoupled Processes

Process A, Block production (fast):

A committee of validators (elected by stake, rotating via VRF) runs a classical BFT consensus (HotStuff family [12]) with block times of 1–2 s. Blocks contain: transactions, inference orders (as commitments), aggregated PoI proofs, slashing events.

Process B, Proof of work (continuous):

Pods submit signed **PoI bundles** per epoch: Merkle roots over (input commitment, output commitment, segment metadata, signatures of all participating shard miners). The block reward of an epoch is distributed to miners in proportion to *confirmed* inference work (after Tier-1 agreement, minus later refuted segments).

Coupling: The voting weight of the validator election draws on two sources: staked coin *and* attested historical inference work (with a decay factor). Useful work thus indirectly secures consensus. Anyone wanting to attack the network must either buy coins massively (market feedback) or perform honest inference work over time (self-contradiction).

3.5.3 Data Availability

Complete prompts/outputs do not belong on-chain (privacy, volume). Only commitments live on-chain; the raw data resides encrypted with the user and (for the dispute window of e.g. 7 days) as erasure-coded fragments with the participating pods, so that bisection games remain executable.

4. Compute Layer: Model Sharding and Pods

4.1 Pipeline Parallelism as the Base Pattern

Tensor parallelism is ruled out over WAN (all-reduce per layer requires sub-millisecond latency). The architecture therefore relies on **pipeline parallelism**: the model is split into k contiguous shards (e.g., layers 1–20, 21–40, …). A **pod** is a chain of k shard miners that jointly execute a complete forward pass. Only activations flow between shards (at batch 1 and hidden dim 8192 in fp8: ~8 KB per token per handoff), that is WAN-viable.

The choice of k is a security parameter. It is tempting to choose k small: fewer, larger shards mean fewer network transitions and thus lower pipeline latency. This optimization is not free, however, because k simultaneously governs three quantities in different directions:

1. **Latency**. The smaller k, the fewer transitions, the faster the pipeline (argues for small k).
2. **Collusion resistance**. The probability that two redundant pods jointly push through a false result falls exponentially with k ($P_{coll} \approx \beta^{2k}$, Appendix B.2). Halving k squares this probability (argues for large k).
3. **Entry threshold and decentralization**, larger shards require more VRAM per node. Once a shard exceeds the VRAM of common consumer cards, only data centers can participate and consumer hardware drops out of the network. This raises β, which additionally amplifies effect (2) (argues for large k).

k is therefore configurable per model version and subject to governance (Ch. 10.3), not to optimization by individual operators. The default choice k = 8 is oriented toward the VRAM of common consumer GPUs; a reduction is defensible only if the resulting collusion bound is explicitly recomputed and the entry threshold assessed.

Latency topology: The networking layer continuously measures pairwise latencies (pings via gossip). The pod formation algorithm (deterministic from block hash + latency graph, see 4.3) preferentially groups geographically close miners into the same pod to minimize pipeline latency, while shard assignment *within* a pod remains random (collusion protection).

4.2 Throughput over Single-Pass Latency

A single forward pass through a WAN pod is slow (on the order of 0.5–2 s pipeline latency plus compute time per token). The architecture compensates via:

- **Micro-batching / pipelining:** While shard 3 computes token t, shard 1 is already processing token $t+2$. For continuous streams (agentic sessions), throughput approaches that of a single shard.
- **KV-cache locality:** Each session's KV cache stays on the shards of the assigned pod (session affinity). Pod changes require a cache rebuild and are triggered only on failure or epoch transition.

- **Speculative decoding:** Small draft models, which fit entirely on individual miners, generate token candidates that the pod verifies in a single batched forward pass, reducing the number of expensive pipeline passes by a factor of 2–4.

4.3 Epochs and Deterministic Assignment

Time is divided into **epochs** (e.g., 1 hour). At the start of an epoch, a seed is derived from the finalized block hash of the previous epoch, which determines via a verifiable random function (VRF):

1. which registered miners receive which shard,
2. how pods are composed (under latency constraints),
3. which inference segments are subject to spot-checking in this epoch.

Since the assignment follows deterministically from public data, every participant can reproduce it independently, there is no central scheduling authority.

4.4 Proximity Within, Distance Between Pods

Latency and collusion resistance pull pod formation in opposite directions. The resolution lies in pursuing both goals at *different levels*:

- **Within a pod**, proximity is optimized: the members of a pipeline should exhibit the lowest possible pairwise latencies (Ch. 4.1), since delays accumulate here across k transitions.
- **Between the two redundant pods**, distance is *enforced*: the twin pod of a segment must originate from a different geographic zone, with diversity requirements also regarding autonomous systems (AS) and, where determinable, operators.

The rationale is not a latency question but the foundation of Tier-1 security: redundancy protects only if the two computations are *independent*. Two pods in the same data center, under the same jurisdiction, or on the same power grid are correlated failure and collusion risks, the assumption of independent faults (Appendix B.2) would be violated, and a single legal intervention could influence both computations at once. The price is low: since the redundant pods do not communicate with one another but merely submit their commitments, their separation costs no pipeline latency and only delays the moment of Tier-1 reconciliation.

5. Tokenomics — The Burn-and-Mint Cycle

5.1 Core Principle

The native coin **MYL** serves three functions: securing consensus (staking), compensating miners (minting), and paying for inference (burning). The cycle is closed:

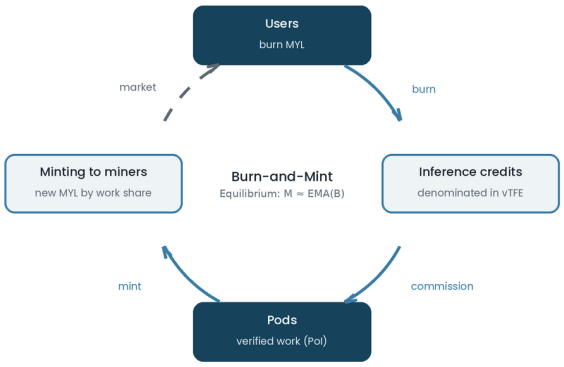

Figure 2: The burn-and-mint cycle. Solid lines are protocol operations, the dashed line is the market.

Training adds no second source of money to this cycle but a feedback loop: if model quality improves, demand rises and with it the burn from which minting is fed. Expenditure on training thus justifies itself through the demand it creates, not through the compute time expended (Appendix B.7).

The protocol issues MYL exclusively to miners; users acquire them on the market. Minting and burning are protocol operations, acquisition is not.

Inference credits are **denominated in computational work** (unit: verified token-forward equivalents, *vTFE*), not in fiat or MYL. The utility price of inference is thus stable in compute units; the MYL price mediates between supply (miner capacity) and demand (credit purchases).

5.2 The Minting Function

Let `B_e` be the MYL volume burned in epoch `e` and `W_e` the confirmed work (in vTFE). The minting `M_e` of the epoch:

$$M_e = \min(B_e \cdot (1 + s), M_{max})$$

Here M_e denotes the minting of epoch e, B_e the exponentially smoothed burn volume, s the subsidy rate, and M_{max} the emission cap.

with:

- `B̄_e` = exponentially smoothed burn volume (EMA over ~30 epochs, dampens manipulation via burn spikes),
- `s` = subsidy rate (bootstrap parameter, starting e.g. at 0.5 and falling toward 0 via a governance schedule),
- `M_max` = hard inflation cap per epoch (residual emission from a fixed total supply, analogous to a halving schedule).

Properties:

1. In equilibrium ($s \to 0$), $M_e \approx B^-_e$: the money supply is long-term **net-neutral to deflationary** (burned coins ≥ minted, since slashing burns are added).

2. In the bootstrap phase ($s > 0$), inflation subsidizes the build-up of miner capacity before demand exists, the same logic as the block subsidy in Bitcoin [13] before meaningful fees.

5.3 Distribution of Minting

```
M_e is split:
  78 %  shard miners       (proportional to confirmed vTFE, after redundancy normalization)
   5 %  pod coordinators   (proportional to coordinated segments)
  10 %  validators         (proportional to stake × uptime)
   4 %  checker pool       (base compensation; bounties come additionally from slashes)
   3 %  protocol treasury  (governance-managed: model updates, audits)
```

Training compensation. Training work (Ch. 7) is financed not from minting but from the protocol treasury and a governance-disableable surcharge on the inference fee. This choice is not arbitrary: financing from additional minting would nearly double net inflation and thereby dilute all holders, whereas treasury and fee surcharge leave the cycle untouched (Appendix B.7).

An upper bound applies to compensation per compute hour: it must not reach that of inference. Otherwise miners shift capacity from inference to training and deprive the network of its only source of revenue. The design sets at most seventy percent of the inference compensation; the value is a governance parameter (Ch. 10.3).

Redundancy normalization: Since every segment is computed by r = 2 pods, each pod receives half the vTFE credit. Miners are thus paid for *useful net work*; the redundancy overhead is priced in, not hidden.

Implementation status. The minting function from 5.2, the distribution from 5.3, and the training compensation cap are implemented in the reference implementation and verified across 10,000 simulated epochs with random values: the sum of distributed shares equals M_e exactly in every epoch (floor rounding per share, rounding remainder closed to the treasury). The computation is entirely integer-based, analogous to Chapter 6.2.

5.4 Credit Pricing

The MYL → vTFE exchange rate is set algorithmically per epoch (EIP-1559-analogous):

$$P_{e+1} = P_e \cdot \exp(\kappa\,(u_e - u^*))$$

Here P_e is the credit price of epoch e, u_e the measured utilization, u^* the utilization target, and κ a damping constant.

- $utilization_e$ = requested vTFE / available pod capacity,
- $target$ = 0.8 (buffer for load spikes),
- k = damping constant.

Under overload, the credit price rises → demand falls, mining becomes more attractive → capacity grows. The mechanism replaces central capacity management with price signals.

5.5 Staking and Slashing Matrix

Actor	Stake requirement	Slash reason	Slash amount
Shard miner	∝ claimed reward capacity	false result (proven via bisection)	100 % of stake
Shard miner	–	unavailability during session	1–5 % (tiered)
Pod coordinator	additional stake	false PoI aggregation	100 %
Validator	BFT stake	double signing / censorship (proven)	30–100 %
Checker	bond per challenge	willfully false challenge	bond

Training segments. The same structure applies to them, but weighted differently: the gain from cheating is smaller, since training compensation is lower, while the damage is greater, for a segment that slips through affects one answer, whereas a gradient that slips through affects the model and thus all future answers. What is raised is therefore not the stake but the **sampling rate**: it acts immediately and costs capacity rather than capital lockup.

Incentive inequality (security condition): A miner cheats rationally only if expected gain > expected penalty. With sampling rate p (Tier 2), stake S, and fraud gain per segment g:

$$p \cdot S > g/p \Leftrightarrow S_{min} = g/p^2$$

Here p is the sampling rate, S the posted stake, and g the gain from a cheated segment. The derivation is given in Appendix B.1.

At p = 2% and g = one segment's reward, S_{min} = 2500 segment rewards; at a capacity of one hundred segments per epoch that corresponds to roughly twenty-five epochs of income. That is the quantitative anchor for the stake parameters and is adjusted via governance to measured fraud rates.

5.6 Attack Vectors and Countermeasures

- **Self-dealing (miner buys own inference to harvest minting):** Unprofitable by construction as long as $M_e \leq B^-_e$ (equilibrium): the attacker burns more than they get back (receiving only their capacity share of minting). In the subsidy phase ($s > 0$), self-dealing is dampened by EMA smoothing and a per-address burn cap.
- **Grinding the pod lottery:** The VRF seed comes from the finalized block of the previous epoch; miner registration closes 2 epochs before assignment (no last-minute injection of prepared identities).
- **Sybil on checker bounties:** A challenge costs a bond; false challenges burn it.

5.7 Issuance Structure and Launch Phase

Chapter 5.2 describes how minting arises but says nothing about how the network begins. This section closes that gap.

Why a start at zero is impossible. The design would call for a fair launch without any initial allocation: all MYL arise from verified work. This fails on a feedback loop long described for stake-based systems [36][37]: a protocol requires a valuable native asset in order to be secure, and must be secure for the asset to acquire value. Work-based systems circumvent this by

converting scarce external resources into coins; in Myelith, however, the work is itself bound to existing stake. Miners must post stake in order to accept work at all (Appendix B.1); without existing MYL nobody can post stake, and without miners no minting arises from which stake could be formed. The stake requirement exceeds the credit requirement of the first users more than a hundredfold and thus alone determines the necessary initial quantity (Appendix B.8).

How the initial quantity stays small. The security condition reads $S_{min} = g/p^2$, depending quadratically on the sampling rate. Raising p during the launch phase reduces the stake requirement drastically: at fifty percent instead of two percent it falls to one six-hundredth. This costs capacity, since every second segment is recomputed, but is bearable in a phase of excess capacity. The rate is lowered on schedule toward its target as the network grows, while the stake requirement rises correspondingly and can be met from ongoing minting.

The initial quantity is therefore measured against the stake requirement of the launch phase under a raised audit rate, not against a chosen target value.

Distribution. The genesis quantity goes exclusively to participants of the preceding testnet, measured by work performed and verified there, plus the treasury share of Chapter 5.3. No pre-sale takes place, and there is no allocation to developers or investors beyond the treasury. This stipulation follows not only from the protocol's self-conception: an issuance against payment with expectation of return would be assessed differently in many jurisdictions than a work-bound acquisition.

No fixed emission cap. It is tempting to fix a total supply or cap minting per epoch. Neither is provided for here, for a reason arising from the construction itself: minting is coupled to the smoothed burn and therefore grows only with demand in any case. An additional cap decouples it from that. Once demand exceeds the cap, work performed is no longer fully compensated; miners leave the network and capacity falls. Model calculations show that a binding cap does not stabilize circulation but brings it to a standstill, since more is burned than minted (Appendix B.8). Scarcity in this system arises not from an upper bound but from the coupling to actual usage.

Early-phase concentration. Those who participate early acquire a disproportionate share of what is minted overall. With work-bound issuance this is unavoidable and is documented for fair launches: even without pre-sale and insider allocation, early participants can amass considerable holdings; an agent-based study finds the concentration to arise regardless of the initial allocation and attributes it to the tradability of the tokens [38]. The remedy is not a different distribution but a flat subsidy curve: the lower the initial subsidy s, the smaller the advantage of early participation. The parameter is subject to governance (Ch. 10.3) and to the invariant of Appendix B.4.

6. Verification — Formal Model

This chapter replaces the verification model of v0.1. What changes is not the principle but its foundation: v0.1 required bit-identical floating-point execution and therefore had to prescribe the order of every operation. v0.2 moves the requirement one level deeper and performs inference **entirely in integer arithmetic**. Since integer addition is associative, bit equality then arises without any ordering prescription, regardless of how the hardware parallelizes. Section 6.3 documents why the obvious alternatives (floating point with fixed ordering, tolerance comparison) were rejected.

6.1 Notation

An **inference segment** σ is a tuple `(x, θ_v, π, y)`:

- `x` = input commitment (hash over prompt chunk ‖ KV-cache root),
- `θ_v` = model version: weights, quantization scheme, and execution specification (6.5),
- `π` = pipeline path (ordered miner list of the pod),
- `y` = output commitment.

Every shard miner `i` on the path signs its transition: `sig_i( h(a_{i−1}) ‖ h(a_i) ‖ σ_id )`, where `a_i` are the activations after shard i. The chain of these hashes is the **computation trace**; it enables bisection without storing activations on-chain.

6.2 Integer Execution as the Basis of Determinism

Non-determinism in neural networks arises not from computational precision but from the **non-associativity of floating-point addition**: for floating-point numbers, $(a + b) + c \neq a + (b + c)$, because rounding occurs after every step. A matrix multiplication sums thousands of products, and the order in which a GPU combines partial results depends on kernel implementation, block partitioning, and runtime conditions. Two honest nodes therefore normally obtain different bits.

Integer addition, by contrast, is associative. A sum of integers yields the same result regardless of the order in which it is formed. If inference is performed entirely in integer arithmetic, bit equality is therefore not a constraint imposed on execution but a property of the arithmetic itself.

That such execution is possible is not an assumption of this work but an established result. Early approaches for transformers replaced the square root in normalization with an L1-norm equivalent [41], building on dyadic arithmetic for convolutional networks [42]. I-BERT quantizes the entire inference, including the non-linear operations GELU, Softmax, and layer normalization, via integer approximations, achieving accuracy matching or slightly exceeding the floating-point reference [18]. I-ViT confirms this for vision transformers [19], and I-LLM extends the approach to large language models [20]. This was preceded by the observation that transformer activations exhibit individual dimensions of strongly elevated amplitude that require separate treatment during quantization [17]. Throughput also favors it: relative to fp32 inference, speedups by factors of 2.4 to 4 are reported [18], since integer units are widely available on common hardware.

Only three stipulations are binding, all part of θ_v:

1. **Fully integer execution.** No floating-point operation in the inference path, including the non-linear functions. Their approximation coefficients and shift widths are part of the model version.
2. **Accumulator width and per-tensor bit width.** A 32-bit accumulator is prescribed. The bit width of weights is fixed per tensor, not globally; the justification follows from Chapter 6.9. With int8 factors the largest product magnitude is 16,129; across typical reduction lengths the margin to overflow remains several orders of magnitude (Appendix B.5). Overflow behavior (saturation) must nonetheless be stated explicitly.
3. **Division exclusively as arithmetic right shift.** This is the only remaining source of platform-dependent results: for negative numbers, flooring division and truncation toward zero differ, and programming languages implement these differently. The arithmetic right shift, by contrast, is identically defined on all common architectures and corresponds throughout to flooring (Appendix B.5).

What is expressly not prescribed: reduction order, block partitioning, kernel implementation, use of matrix units. The hardware may parallelize as it sees fit. This removes the throughput loss that a forced operation order would entail, and likewise any preference for particular hardware classes. The latter is no minor point: a mandate favoring high-precision floating-point accumulation would disadvantage consumer accelerators, on which that path frequently runs at only half rate, while data-center hardware knows no such penalty. It would thus run counter to the network's decentralization goal.

6.3 Rejected Alternatives

Two obvious designs were examined and rejected. The reasons are documented because they bear on related work; the evidence is given in Appendix B.5.

Floating point with enforced ordering. This is the approach of RepOps [15] and comparable libraries [16]: a fixed reduction order establishes bitwise reproducibility across different hardware. The approach is proven but has three drawbacks for our case. First, restricted parallelization costs throughput; substantial overheads are reported depending on the setup. Second, it presupposes that the hardware follows a uniform floating-point standard, largely true for single precision, but not for half precision and the matrix units designed for AI: their rounding behavior, accumulator widths, and normalization points already differ between the architecture generations of a single vendor [21]. Third, RepOps expressly covers only the single-device case; reproducibility across multiple nodes with pipeline or tensor parallelism is named as future work [15], and precisely this case obtains in Myelith.

Tolerance comparison instead of bit equality. Rather than requiring equality, one could accept a distance below a threshold τ, as locality-sensitive commitment schemes propose for auditing individual providers [14]. Three findings argue against it. Computational noise accumulates across chained execution to the point where the results of two honest nodes diverge, after a few layers, as much as manipulated ones do from unmanipulated. The required separability is moreover not robust: under violated distributional assumptions the requirement rises manyfold. And decisively: a tolerance band is adaptively attackable. Whoever knows the verification criterion tailors the manipulation to it; with structure-based commitments it suffices to compute the audited components correctly and falsify the rest, which in simulation remained undetected across ten subsequent layers.

A third route would be to emulate floating-point operations entirely in software. It yields bit-exact results without hardware requirements but replaces every single operation with a sequence of integer instructions. If integer arithmetic is used anyway, it is more consistent to execute the model itself in integers than to reconstruct floating point on top of them.

Integer execution avoids all three problems: it requires no ordering prescription, knows no tolerance region with which an attacker could align, and reconstructs no foreign arithmetic.

6.4 The Three Verification Tiers

Tier 1, Deterministic redundancy (immediate, cheap).

Every segment is computed by r = 2 independently assigned pods. If the commitment hashes match at all trace positions, the segment is provisionally confirmed. The comparison is binary and parameter-free; there is no threshold to calibrate, attack, or shift by governance.

Timing of the comparison, two delivery modes. The comparison may occur before or after delivery; both variants are selectable per request:

- **Optimistic delivery (default).** The response of whichever pod finishes first is delivered immediately; the comparison occurs asynchronously and takes effect through slashing and reversal of the vTFE credit. Latency equals that of a single pod; security is retrospective.
- **Confirmed delivery (selectable, surcharge).** The response is withheld until the twin pod agrees. A manipulated result does not reach the user unless both pods collude. The price is latency and a fee surcharge.

The choice belongs to the user: for research, retrospective sanction suffices; for an agent decision with financial effect (Ch. 8), the preventive variant is appropriate.

Tier 2, Optimistic sampling (delayed, targeted).

Checkers recompute VRF-drawn segments (~1–3% of volume). On deviation, the bisection game starts (6.6).

Tier 3, zkML anchors (rare, expensive, maximally secure).

An optional premium path for results with financial effect, and an upgrade path once zkML systems become efficient enough. Integer execution favors this path, since arithmetic circuits over integers are considerably simpler to formulate than over floating point.

6.5 Execution Specification as a Protocol Property

The execution specification is part of θ_v and thus consensus-relevant. It comprises the quantization scheme (bit widths for weights and activations), accumulator width, overflow behavior, the coefficients of the integer approximations of non-linear functions, the rules of dynamic quantization, and the stipulation of the arithmetic right shift.

A miner deviating from this specification (computing at lower bit width or skipping layers) saves real cost and degrades real output quality. Under bit equality this is no borderline case but immediately visible, since any deviation drives the hashes apart. Likewise: any compression of activations on the wire (Ch. 10) must be protocol-defined and identical for both redundant pods, since it would otherwise form part of the execution specification.

Not part of the specification, and thus freely chosen, are kernel implementation, parallelization strategy, block sizes, and memory layout. Here lies the degree of freedom that lets heterogeneous hardware participate without competitive disadvantage.

6.6 The Bisection Game

If a checker claims segment σ is false, an interactive protocol with O(log L) rounds runs (L = number of shard transitions):

```
1. Checker submits its own trace h(a'_0..k); let the first divergent transition be j
2. On-chain, only layer group j is decided:
   - Miner j reveals a_{j-1} (erasure-coded fragments from the DA layer)
   - A validator committee executes ONE shard forward according to theta_v
   - Comparison with h(a_j): miner or checker loses
3. Loser: slash; winner: bounty
```

Attribution of fault is unambiguous because the result is canonical: there is exactly one correct `a_j`, and the comparison is a hash equality without discretion. Integer execution further simplifies arbitration, since validators need no special hardware and no certified kernel implementation; a reference execution in integer arithmetic yields the same result on any platform. Costs remain constant: a single shard forward on roughly seven validators, independent of segment length and never incurred in regular operation.

6.7 Control Segments

The three tiers sanction fraud either by comparing two computations or by retrospective sampling. Both presuppose that the twin pod computes honestly or that an attacker is conspicuous repeatedly. For the one-off intervention of an attacker controlling both pods, a gap remains.

Control segments (canaries) narrow it. The network maintains a stock of segments whose correct result is already available because they were computed under controlled conditions. Gateways inject such segments into the regular order stream at a share γ. To the executing miner they are indistinguishable from genuine user requests; their verification is trivial, since the target commitment is on record.

The security gain lies in the attacker's **uncertainty**: since they cannot know for any segment whether it is a control, even the first manipulation attempt carries a detection risk of γ, regardless of whether they control both pods. At γ = 2% and full stake loss upon detection, the expected value of a single attack is already negative, provided the attacker values their stake at all.

Three construction requirements are essential:

1. **Indistinguishability.** Control segments must be drawn from the real prompt distribution and be unremarkable in timing, length, and context profile. If a miner can identify them by statistical features, the mechanism is void (Ch. 10, point 5).
2. **Stock renewal.** A static pool becomes recognizable over time. An obvious route is admitting completed segments fully audited under Tier 2.
3. **Cost honesty.** Control segments are pure overhead; γ enters the cost structure directly and is a governance parameter (Ch. 10.3).

6.8 Security Argument (Sketch)

Under the assumptions that (a) $\leq f < 1/3$ of validator votes are Byzantine, (b) at least one honest checker audits every sample, and (c) the DA layer delivers activations during the dispute window:

- **Soundness:** A false segment survives the dispute window only if both redundant pods compute identically wrong results (collusion across the VRF lottery, $P \approx \beta^{2k}$, Appendix B.2), it is not drawn in the sample ($P = 1-p$), and it hits no control segment ($P = 1-\gamma$). These events are independent; total risk is multiplicative. Since the comparison is binary, no tolerance region exists in which a manipulation could hide.
- **Liveness:** If a shard miner fails, the pod's standby miner takes over (k+2 members, 2 in reserve); session loss occurs only if more than two simultaneous failures hit the same pod.

6.9 Empirical Test on a Reference Implementation

The assumption of 6.2, that a language model can be executed entirely in integer arithmetic without appreciable loss of output quality, has been tested on a reference implementation since version 0.2 of this paper. The basis is two dense models under a permissive license, of 0.5 and 7 billion parameters; weights in eight bits with per-channel power-of-two scales, activations in sixteen bits with calibrated per-layer scales.

Result. Measured on a fixed excerpt of WikiText-2, by identical method on the same sequences:

Model	Integer path	Floating-point reference	Gap
0.5 billion parameters	15.27	14.95	+2.11 %
7 billion parameters	8.78	8.68	+1.14 %

Both stay below the five percent threshold this project uses as its acceptance criterion. Agreement of the most probable token with the floating-point reference is 89.7 percent on the smaller model (394 of 439 positions). All percentages in this section are formed from the unrounded measurements and cannot be reproduced exactly from the table values, which are rounded to two decimals.

Determinism is confirmed. Twenty-five independent runs in five groups yield bit-identical results. Instrumentation of the entire forward path finds no floating-point operation. The

property on which this chapter's verification rests is thus demonstrated on a real implementation, not merely argued.

Two computation paths, the same bits. A comparison of two different implementations of the same operation shows this more clearly than any repetition of the same code: a scalar dot product and a vectorized one using the processor's SIMD units. Vectorization changes the reduction order, which is precisely what produces divergence in floating point. Measured, both paths yield the same hash over all generated tokens, at 27 percent (0.5B) and 43 percent (7B) higher throughput. The assurance of 6.2, that hardware may parallelize as it likes, is therefore no longer a claim.

The demonstration extends to distributed execution as well. A four-stage pipeline of genuinely separate operating-system processes, communicating over real TCP connections, produces the same token sequence as the single-node reference for a repeated identical prompt and is deterministic across independent runs. Under artificially injected network stress, that is a 100-millisecond delay per stage transition, connection drops with retry logic via duplicate detection, and a full restart of individual nodes with an idempotency check, the result remains bit-identical. This tests the assumption from 6.2 for the first time not merely within one process but across the network protocol between separate processes, so far however on a single machine over loopback connections and not across physically separate hosts. The demonstration across different machines and operating systems is outstanding and is carried as an open question in Chapter 11.

Where the cost arises, and where it does not. More informative than the final figure is the decomposition of the error. Applying the same quantization stepwise in floating-point arithmetic separates what the *method* costs from what the *implementation* costs. On the larger model:

Stage	Perplexity	Gap
Floating point, not quantized	8.68	reference
Weights int8 per channel, arithmetic in floating point	8.74	+0.69 %
Additionally activations int16, arithmetic in floating point	8.75	+0.84 %
Fully integer path	8.78	+1.14 %

The third row is the **floor of this quantization scheme**: the price the chosen representation costs, regardless of who implements it. It stands at 0.84 percent. A different scheme, say with finer grouping of scales, would have a different floor; the comparison here is against the floor of our own. The fourth row adds everything integer execution itself contributes, namely lookup tables for the non-linear functions, rounding at every rescaling, and the coarseness of the scale grid. That implementation share amounts to 0.30 percentage points.

The most expensive error of this work was mistaking a gap for a limit of the method. The same measurement on the same model stood at +377 percent at one point and at +8.3 percent later. Both figures looked like a property of the method and were implementation errors, three of them, all from the same family:

- Quantization to eight bits saturated silently above magnitude 127. Sixteen of 129,024 bias elements were affected, a share no sampling would have found.
- Normalization aligned its variance sum against the *smallest* channel shift. With a wide shift span this erases the finely scaled channels from the sum: an ordinary channel contributed 2 instead of 160,000, and normalization came to rest almost entirely on the coarse outlier channels.
- The addition onto the residual stream rescaled and clamped both operands separately before adding them. At a cancellation this destroys the value completely, because both operands can be large while only their sum is small.

The common denominator is the property described in [17]: transformer activations carry individual channels of far greater amplitude than the rest. Per-channel power-of-two scales therefore span a wide range, and **any operation that aligns to the wrong end of that range, or clamps more than once, erases the finely scaled channels.** Whoever implements integer inference should, at every point where values of different scales meet, align against the largest shift, accumulate at sufficient width, and saturate exactly once at the end.

What follows methodologically. The search was decided not by reading code but by reference simulations: the same quantization scheme, executed in floating point. Run separately for weights and for activations, each stayed below one percent while the implementation was off by a three-digit margin; a later joint measurement produced the 0.84 percent of the table above. Both times the question was answered before a single line was changed: the method was not failing, our own code was. Whoever implements integer execution should write this simulation first. It is cheap, and it prevents weeks spent improving a method that already works.

A clear priority for effort follows at the same time: invest not in the representation of weights but in the precision of intermediate computations. Several obvious routes demonstrably remained without effect, among them finer weight quantization by the method of [44], an increase in weight bit width, and a broadened calibration corpus. All of them improve the scheme, which already stands at 0.84 percent.

The quality measure you choose determines what you see. Between two states of the same implementation the perplexity gap halved, from 4.3 to 2.11 percent. Top-1 agreement moved by 0.4 points in the same step, from 89.3 to 89.7 percent. Free generation, by contrast, jumped from zero to two of five prompts running token-identical to the floating-point reference. The reason is structural: the top-1 measurement checks each position against a fixed context and is therefore insensitive, whereas in free generation every deviation propagates into everything that follows. Perplexity sits between them. Anyone reporting a single figure should say which one.

The execution specification must be obtained from the weights, not from the architecture description. Several assumptions plausibly derived from the model description proved incorrect: the presence of bias terms in the attention projections, the arrangement of key and value heads under grouped-query attention, and the value range actually occurring at the input of the exponential function. None of these concerns

determinism, but each would have produced wrong results. For Chapter 10.1 it follows that the specification is a measurement result and cannot be derived from a model's documentation.

Not every tensor tolerates the same bit width. Where a model carries input embedding and output projection on the same weight, the error tolerance of the two uses differs considerably. In the embedding lookup the quantization error acts additively on a residual stream of far greater amplitude and remains inconsequential; in the output projection the same error directly decides the ranking of tokens. Bit width is therefore to be fixed per tensor and forms part of θ_v. Determinism is unaffected, since a higher bit width does not alter associativity; only memory requirements are concerned.

The measuring instrument must be measured too. This chapter's soundness claim rests on two pods comparing their results bitwise. The protocol therefore forms their commitment over the activations (6.1), and the reference implementation showed why that choice is not self-evident. The digest initially used there to compare two runs covered only the *chosen* tokens, not the numbers the choice arose from. An argmax over more than 150,000 logits changes only when their ranking changes: in a control measurement the tokens stayed the same while deliberately shifted bytes in the tensor did change the numbers. A second case of the same kind certified a computation path the tested platform does not possess. Neither error lay in the model; both lay in the statement about the model. **Whoever implements this design should record two things:** a segment's commitment must be formed over the computed quantities and not over the decision derived from them, and a conformance run must record which computation path actually ran.

Limits of this result. Measurement was performed at two sizes, 0.5 and 7 billion parameters. The scale intended in this paper, 100 billion to one trillion, therefore remains unestablished. The common assumption that larger models are more robust to quantization fits both data points, since the larger model performs better. It still deserves a caveat: the only data point that ever appeared to contradict it in this work was an implementation error. Likewise outstanding is the demonstration that bit equality holds across different hardware classes and across physically separate hosts; the runs reported here were performed on a reference implementation without accelerators, and the multi-node runs on a single machine (Ch. 11, points 2 and 3).

6.10 What This Design Costs

For the sake of honesty, the counter-calculation. Integer execution avoids the throughput loss of an ordering prescription but requires a **quantized model**. The protocol thereby binds itself to a model class whose quality relative to the floating-point reference must be demonstrated. The available literature is encouraging but not conclusive: eight-bit quantization is broadly established [18][19], its extension to large language models is more recent [20] and not comprehensively validated for models at the scale intended here. Should integer execution prove to cause noticeable quality losses at the target model size, the basis of this chapter would require reassessment (Ch. 10, point 1).

Furthermore, the determinism property rests on the completeness of the specification. Overlooked platform-dependent operations (in the behavior of integer matrix units, at saturation boundaries, or through compiler transformations) could reintroduce non-determinism. Unlike with floating point, however, such cases are enumerable and testable through conformance suites with finitely many test vectors; they require no restriction of parallelization.

What persists even under bit equality: an attacker controlling **both** assigned pods can produce a consistently false result. Against this act the lottery assignment (Ch. 4.3), the enforced zone diversity of twin pods (Ch. 4.4), sampling, and control segments, each with its own independent probability. This case is not excluded; for applications where even that is intolerable, Tier 3 exists with cryptographic rather than probabilistic guarantees.

7. Training and Model Development

A network that operates a language model should also be able to advance it. Otherwise the model ages while capacity grows, and the network remains permanently dependent on external training runs. This chapter describes how training is fitted into the existing architecture, which requirements this imposes, and where the limits of the approach lie.

The starting position is more favorable than expected: the integer execution of Chapter 6 carries over unchanged to the backward pass, since gradient computation is likewise associative. Verified computation alone, however, does not suffice. Unlike inference, where correctness is the entire question, a bit-exact correct training computation may rest on unsuitable data or act in a harmful direction. Sections 7.3 to 7.5 address this gap.

7.1 Training as a Subordinate Work Class

Inference has unconditional priority. It generates the fees that finance the network, and users expect response times that no background process may impair. Training therefore runs as a second, subordinate work class in the residual capacity.

The scheduler assigns training segments only to pods whose utilization in the preceding epoch fell below a threshold, and caps the share at a **base rate** γ_train. A value in the range of five to ten percent of free capacity is the sensible starting point. The reference quantity matters here: γ_train is measured against *free* capacity, not against the network's total output. At seventy percent utilization, ten percent of free capacity corresponds to roughly three percent of total output and thus to the magnitude carried by the treasury of Chapter 5.3. The upper bound follows not from a fixed number but from measured inference demand: if utilization rises above the target of Chapter 5.4, training is throttled before credit prices increase. This rules out training displacing inference capacity.

What this capacity achieves. For a model in the 24-billion class at a base rate of ten percent, a network of 5,000 miners reaches roughly one billion training tokens per day, one of 50,000 miners roughly nine billion (Appendix B.6). A fine-tuning

run is thus achievable in days. A full pre-training run, requiring trillions of tokens, is not, and will not be even under substantial growth. The network can advance a model, not create one; dependence on an externally pre-trained base model is permanent, not merely initial.

7.2 Local Loss Blocks Instead of Global Backpropagation

Integer backpropagation encounters an overflow problem: error terms grow with each layer traversed backward, and with eight-bit weights they exceed the 32-bit range after only a few layers [22]. Two methods resolve this, and both fit the present architecture.

First, the **block scaling** of NITI [23]: after each layer the error vector is divided by a common power-of-two factor whose exponent is carried separately. The factor follows from the magnitude maximum and is therefore order-independent; it is applied as an arithmetic right shift, that is, with exactly the operation Chapter 6.2 already mandates. Over forty layers no overflow occurs (Appendix B.6).

Second, **local loss blocks** [24]: the network is divided into segments with their own loss functions, so that gradients do not leave the segment. For Myelith this is more than overflow avoidance: placing the block boundaries on the shard boundaries eliminates the backward pass across the pipeline entirely. No additional network traffic arises, and verification remains local, with a shard pair checking its own gradient. The price is a possibly poorer solution than global backpropagation. [24], however, attributes such a gap to integer-only arithmetic rather than to the local loss blocks; how large it is for language models remains open (Ch. 11, point 3).

Tested, not merely cited. Both methods come from the literature; their combination with the quantization scheme used here does not. It was therefore simulated on a model of 0.5 billion parameters on WikiText-2, with a held-out set that is never trained on. Over 200 steps:

Variant	Loss on held-out text	Gap to floating-point reference
Floating-point reference	3.047 → 2.980	
Integer scheme, round to nearest	3.069 → 3.871	+29.9 %
Integer scheme, stochastic rounding	3.077 → 2.999	+0.67 %

The difference between the two integer rows is the rounding mode, nothing else. At a learning rate of 1e-5 a single step moves a weight by a median of six millionths of a grid level. Round-to-nearest discards every one of those changes, the model stands still, and what looks like a learning curve is memorization of the training set. Stochastic rounding carries them through correctly on average [45]. For a design that treats training as a class of work, this is not a detail but the condition under which it functions at all.

Randomness in a system that lives on bit equality. Stochastic rounding introduces randomness exactly where Chapter 6 demands determinism. The contradiction dissolves once the randomness is not a *source* but a *function*: the draw follows from (layer, step, index, epoch seed) via a counter-based generator, with no internal state and no dependence on order. Two nodes with the same gradient thereby obtain the same new weight state, bit for bit, and the recomputability of 6.1 remains intact. That the alternative does not hold was measured in passing: the random generator of a widely used training library produced different results on the accelerator of the same machine, in two processes, from an identical seed. A stateful generator is unusable for a consensus network.

The weight state stays integer. So that a training step leaves no floating-point state behind, every weight carries an integer master with additional fractional bits; the forward path arises from it by stochastic rounding to eight bits, and the update is an exact integer addition. Measured, this costs practically nothing against a floating-point master, namely 0.08 percentage points (+0.75 % versus +0.67 %). The additional bits below the eight-bit level *are* the quantization remainder: it is not discarded but continues to act at the next step. What the literature treats as a mechanism of its own (error feedback, [46][47]) falls out here as a side effect of word width. The required width follows from the step size: at the learning rate above, eighteen fractional bits suffice at the median, twenty-five are recommended with reserve, and the sum over millions of contributions calls for a 64-bit accumulator (Appendix B.6.11).

7.3 Data Provenance Instead of Data Assessment

The hardest question in training is not whether the computation was correct but whether the data were legitimate. A miner feeding in poisoned text computes bit-exactly correctly and nonetheless produces a shifted model. The bit comparison of Chapter 6 does not reach this.

An assessment of data content is ruled out: it would be subjective and thus precisely the scope for judgment that the protocol deliberately avoids elsewhere (Ch. 2). Myelith therefore verifies not content but **provenance**. The method is borrowed from the blockchain-based federated learning literature [30]–[33] and transferred here to open network operation (cf. Ch. 2).

The protocol maintains a list of canonical corpora, each anchored by a Merkle root in consensus. A training segment references not raw data but a Merkle proof: the text passage stands at a specific position in the corpus with root R. Verification thereby becomes objective again and exactly as verifiable as an inference. A miner cannot smuggle in their own data, because they cannot produce a valid proof for non-existent positions.

The cost is low, provided segments are assigned in batches: a single proof over a corpus of one billion documents costs just under twelve percent overhead, whereas a joint proof over 256 contiguous segments costs less than half a percent (Appendix B.6).

Selection remains an attack surface. Whoever cannot forge data can still select. An attacker with forty percent of capacity would, given free choice, hold forty percent of influence over data composition. Data assignment therefore likewise proceeds **by VRF**: which pod processes which corpus sections follows from the epoch seed, not from the miner's choice. All that remains to them is refusing assigned segments, which costs

compensation and becomes visible through the refusal rate. Residual influence thereby falls to a few percent (Appendix B.6). This requirement is constitutive, not optional.

7.4 Aggregation and Adoption

Robust aggregation. The gradients of many pods must be combined into one update. The obvious mean is unsuitable: a single extreme contribution shifts it arbitrarily, and as few as five percent Byzantine pods produce a marked distortion (Appendix B.6). Myelith therefore aggregates via the **median** [35]. Its breakdown point lies at fifty percent and thus coincides with the Byzantine bound already assumed [35]; trimmed means, by contrast, fail at a one-third attacker share. The methods originate in the literature on Byzantine-robust federated learning [34][35] and are adopted here unchanged; their known weakness under strongly non-identically distributed data applies here as well. The median requires only comparisons and thus remains deterministic and verifiable within the model.

Handling stale gradients. Pods compute at different speeds. Every gradient therefore carries the model state on which it was computed; contributions older than a fixed number of steps are discarded. Model calculations show that moderate delay slows convergence but does not prevent it (Appendix B.6); where the practical limit lies remains to be measured.

Adoption of new weights. The three-stage process of Chapter 10.2 applies, with two additions following from the analysis. First, the **hold-out set of the shadow phase is drawn by VRF from the corpus only after training concludes** and then disclosed. A benchmark known in advance otherwise permits considerable apparent progress without genuine improvement. Second, the assessment includes **regression tests**: an update that degrades existing capabilities is rejected, even if it improves new ones.

Replay share against forgetting. Continuous training on new data lets existing capabilities decay. Without countermeasures a substantial part is lost; a replay share of roughly fifteen percent from the existing corpus markedly limits the loss (Appendix B.6). This share forms part of the VRF-controlled data assignment and is thus not influenceable by miners.

7.5 Model Growth

Fine-tuning maintains a model but does not enlarge it. Between fine-tuning and pre-training lies a third path suited to a growing network: incremental enlargement of the existing model.

Function-preserving expansion. Methods such as Net2Net [25] and bert2BERT [26] enlarge a model without altering its function: neurons are split, new layers initialized as identity. The enlarged model behaves identically to its predecessor immediately after expansion. Two consequences follow for Myelith. First, a growth step is **activatable without quality risk**, since behavior does not initially change; improvement arises only from the subsequent training. Second, the expansion is a deterministic transformation of the weights and thus **bit-exactly verifiable** like any other computation. The new model version θ_v+1 follows reproducibly from θ_v and the growth operator; both are anchored on-chain.

In integer form the expansion is not approximately but exactly function-preserving. In floating point, splitting a neuron leaves a rounding remainder, and the check "behaves like its predecessor" is a tolerance comparison. In integer form both fall away. If the outgoing column is not halved but **split**, that is $a = \lfloor m/2 \rfloor$ and $b = m - a$, then $a + b = m$ holds for every integer, odd or even, and nothing is rounded. Measured, the output after expansion is bit-identical to the output before, maximum deviation 0.00e+00. A first attempt to halve via the scale instead was off by 1.24e-03; the difference between "almost equal" and "bit-identical" is here the difference between a statement and none. **The acceptance criterion of a growth step is therefore a digest comparison**, the same check the protocol demands for every other computation anyway.

Symmetry breaks without artificial noise. Two exact copies of a neuron would receive identical gradients and stay identical; the new capacity would be dead, and Net2Net solves this by adding noise. In integer form none is needed: the split separates a and b by exactly one least significant bit at every odd entry, in a check at seven of the sixteen entries examined, that is wherever the source value was odd. The stochastic rounding of 7.2 additionally separates the incoming rows. Both mechanisms are deterministic and recomputable, and both fall out as side effects of decisions taken for other reasons anyway.

Cost. A growth step requires roughly one third of the tokens a pre-training run of the same size would cost, since prior knowledge is preserved (Appendix B.6). The literature reports savings of up to roughly fifty percent [28] and, in one case, 54.6 percent against conventional pre-training [29]; the earlier work [27] does not quantify the gain.

Structural coupling. Depth growth adds layers, which in the pipeline means **additional shards**. More miners enable more shards, more shards carry more layers. Network and model growth are thus linked not merely in time but architecturally. A welcome side effect: the collusion bound β^{2k} of Appendix B.2 improves as k rises, so growth also raises security.

Timescale, soberly considered. A network of 500 miners cannot grow; the first step would lie beyond seven years. With 5,000 miners it takes roughly nine months, with 50,000 miners roughly one month, later steps correspondingly longer (Appendix B.6). Growth is thus not a continuous process but a rare event on the scale of years, bound to substantial network size. Its timing becomes a governance decision: growing too early degrades quality per parameter, growing too late wastes capacity.

7.6 What This Design Leaves Open

Three points remain unresolved and are named here rather than circumscribed.

Financing produces misaligned incentives in every variant. Training burns no credits and therefore generates no burn from which compensation could be derived. Compensation by compute time rewards training regardless of whether it benefits the model, creating the same misalignment the protocol avoids for inference. Outcome-dependent compensation, by contrast, would be subjective and attackable. The present design finances training from the protocol treasury (Ch. 5.3),

supplemented by a governance-disableable surcharge on inference fees, and forgoes any share of minting. That is a compromise, not a solution: it bounds the misalignment by a budget rather than removing it.

The combination of methods is established in the small, not at scale. Integer training is established [23][24], model growth is established [25]–[29]; their combination with the quantization scheme of this paper is, since the simulation of 7.2, no longer unproven, but the proof is small: 200 steps on a model of 0.5 billion parameters, one corpus, one learning rate. What follows is a feasibility statement, not a statement about convergence over billions of tokens. Moreover, the literature evidence for integer training still comes from the image domain with comparatively small networks; for transformers at the scale intended here no demonstration exists.

Behavior under open network conditions is unknown. All literature on progressive growth stems from centrally controlled runs with uniform hardware, uninterrupted schedules, and free choice of data. Whether the same methods work under heterogeneous capacity, interrupted runs, and VRF-assigned data remains open.

8. Agent Layer

The agent layer turns the inference network into a system capable of acting: an agent plans across multiple steps, invokes tools, and can trigger transactions. In doing so it leaves the domain in which the verification model of Chapter 6 holds, since that model rests on the reproducibility of computations. The outside world is not reproducible. This chapter describes where the boundary lies, how it is made visible, and how damage beyond it is contained.

8.1 The Limit of Verifiability

Tier 1 compares the results of two pods for bit equality. If an agent invokes a web search or an external interface, both pods receive different answers, since they query at different moments and the data change. The comparison then fails without any error having occurred.

The protocol resolves this by removing tool results from the computation and making them an **input**: a gateway retrieves the result once, stamps it with time and signature, and passes it to both pods as identical text. The signature enters the computation trace and is thus verified along with everything else. What is verified is the *processing* of the answer, not its correctness.

From this follows a distinction made visible to the user:

- **Deterministic tools** return reproducible answers: queries of the network's own ledger, calculations, access to corpora anchored in consensus (Ch. 7.3). They are fully verified like any other computation.
- **External tools** return non-reproducible answers: web search, market data, third-party interfaces. Their answer is attested but not verified; the protocol testifies *that* a particular gateway received this answer at a particular time, not that it is accurate.

For external tools a trust anchor therefore exists at the retrieving gateway. Multiple retrieval by independent gateways mitigates this where the answer is stable, but fails for continuously changing data. This limitation is named, not concealed.

8.2 Session Contracts and Damage Containment

An agent able to trigger transactions turns a computational error into financial loss. The residual case of Chapter 6.10, a manipulated segment that survives all checks, only becomes truly expensive here. The protocol's answer is not to exclude the case but to bound its effect.

Every agent session runs under a **session contract** with four enforced limits:

1. **Total budget** in credits and, where transactions are permitted, in MYL.
2. **Per-transaction limit**, independent of remaining budget.
3. **Recipient whitelist**: addresses to which payment is possible at all.
4. **Time window**, after which the session expires.

What matters is where these parameters reside: **in the contract, not in the model's context.** They are neither readable nor alterable by the agent. No text the agent processes can influence them; enforcement occurs at transaction execution by consensus.

To this is added a **coupling of amount to security tier**: transactions above a threshold set in the contract execute only if the underlying segment was computed in confirmed delivery mode (Ch. 6.4), where both redundant pods had to agree before the result reached the user. Permitting larger amounts is paid for in latency and fees.

8.3 Handling Injected Instructions

When an agent processes third-party content, that content may contain instructions posing as a user request. This problem is known and unsolved; filter-based approaches are considered unreliable, since the checking mechanism is subject to the same attack surface as the model.

Myelith therefore follows the approach of architectural separation as described by the dual-LLM pattern [39] and its elaboration in CaMeL [40]: the planning component never sees third-party content, the processing component cannot invoke tools, and retrieved data cannot influence control flow. For the present protocol a natural reinforcement follows: permissions reside in the session contract anyway and thus beyond the model's reach (8.2). An injected text can deceive the agent but can neither raise its budget nor add a recipient.

The problem thereby shifts from security to output quality: a deceived agent can make a poor decision within its limits but cannot act beyond them. This is the strongest available claim; complete defense is expressly not asserted by the works cited [40].

8.4 Chaining of Steps

An agent works iteratively: reason, invoke a tool, reason further. Each step is a separate inference segment with its own verification. So that the *sequence* also remains verifiable, each segment references the output commitment of its predecessor. This produces a chain with the same structure as the computation trace within a segment (Ch. 6.1), one level higher.

What becomes verifiable is not only whether each step was computed correctly but also that no steps were omitted, inserted, or reordered. Termination conditions (maximum number of steps, budget exhaustion, goal attainment) reside in the session contract and are enforced by consensus, not by the agent itself.

A session's persistent memory, such as a vector store for agent recall, is operated as its own work class within the network and is subject to the same provenance requirement as training data (Ch. 7.3): whatever enters the store must be traceable to a verified segment.

8.5 Responsibility

For the case where an agent causes harm, the protocol holds no technical remedy, and none is claimed. What it does provide is complete traceability: from the segment chain and the attestations one can reconstruct which pod computed which step, which tool answers were received, and which gateway attested them.

What it does not provide is any assurance that the agent decides correctly. It may happen that every party acted correctly, the protocol functioned without fault, and harm nonetheless arose because the model made a poor decision or an external answer was wrong. Whoever grants an agent powers of disposition bears that risk.

The confidentiality classes of Chapter 9.3 apply here by analogy: agents with transaction rights are suitable for matters whose possible harm does not exceed the budget set, and unsuitable wherever a wrong decision cannot be bounded by a budget.

9. Confidentiality and Risk Model

9.1 Where the Confidentiality Boundary Runs

Myelith is an open network of other people's machines. The decisive property users must know: **shard miners see the activations of the segments they compute.** Activations are not innocuous intermediate values, the input text can be reconstructed from them to a considerable degree. Whoever computes a segment can therefore, in principle, learn what it is about.

This property is not an implementation gap but follows necessarily from third-party hardware executing the model. It cannot be encrypted away as long as computation occurs on plaintext.

9.2 What Edge Encryption Achieves, and What It Does Not

All connections between user, gateway, and the endpoints of the pipeline (first and last shard) are end-to-end encrypted, as are the activation streams between shards. This excludes a real and not insignificant class of adversaries: network observers, intermediate nodes, compromised gateways, and miners not themselves involved in the segment in question. Since gateways only forward and do not compute, this removes a collection point at which the entire plaintext traffic of many users would otherwise converge.

What encryption expressly does **not** achieve: protection from the participating shard miners themselves. Processing the content *is* their task.

9.3 Risk Classes for Users

In place of imprecise assurances, the protocol states explicitly what it is suitable for:

Class	Examples	Suitability
A. Public	Public documents, research, code from open repositories, creative work	Suitable. No loss of confidentiality, as the content is public anyway.
B. Internal, low sensitivity	Internal notes, non-critical business processes	Conditionally suitable. Pod composition changes each epoch; no single miner sees more than fragments.
C. Confidential	Personal data, health and financial data, trade secrets, legal advice	**Unsuitable.** Do not use until hardware-backed confidentiality exists.

Segmentation additionally dampens the risk of class B: a session is split into segments whose assignment changes each epoch, so that a single miner typically sees only fragments. That is an impediment, not a guarantee: several colluding miners can reassemble fragments.

9.4 Path to Higher Confidentiality

Two routes are foreseeable, both with drawbacks that belong stated transparently:

- **Trusted execution environments (TEEs)** on the miner side would enable class C but introduce trust in chip manufacturers. A centralizing factor that contradicts the network's basic idea, and an attack target with a documented history of breaches. Conceivable as an *optional* capacity class with its own compensation, chosen deliberately by users, not as a network default.
- **Homomorphic encryption** would solve the problem fundamentally but is orders of magnitude too slow for models of this scale. A matter for observation, not a basis for planning.

Until then, the classification in 8.3 applies unchanged. A system that clearly states its limits is preferable to one that suggests confidentiality it cannot deliver.

10. Governance and Model Provenance

10.1 The Model as a Commons

Since the weights necessarily reside on miner hardware, the network model must be open-weight. The genesis state references an existing model via the Merkle root of its quantized weights.

Requirements on the base model. Four criteria follow from the architecture, and they constrain the choice more than mere availability:

1. **Permissive license.** Apache 2.0 or MIT are required. Licenses with user-count caps or geographic restrictions are ruled out, since an open protocol neither knows nor can limit its user count. Apache 2.0 is preferable to MIT, as it includes an explicit patent grant.
2. **Dense rather than mixture-of-experts.** MoE models route each token dynamically to varying experts, so the data path differs per token. The fixed pod chain of Chapter 4 presupposes a constant path. Dense models are more costly per parameter but architecturally compatible.
3. **Moderate layer count at high quality per parameter.** Every shard transition costs network latency. Models that achieve their quality through width rather than depth are at an advantage for WAN pipelines.
4. **Integer quantizability.** The model must be fully executable in integer arithmetic, including the non-linear operations (Ch. 6.2).

State of availability. The third and fourth requirements are satisfiable but not readily available. For dense models of the 24-billion class under Apache 2.0, INT8 quantizations exist that roughly halve memory requirements and roughly double matrix-multiply throughput. These, however, quantize only the linear operators within the transformer blocks; softmax, layer normalization, and the activation functions remain in floating point. For Myelith this does not suffice: if even one floating-point operation remains in the path, the determinism property of Chapter 6.2 is lost.

Producing a fully integer quantization along the lines of [18] and [20] is therefore necessary preparatory work before the genesis block, and at the same time the first test of the design's viability (Ch. 11, point 1).

Who warrants the quantization. The quantized model is part of θ_v and thus consensus-relevant. Its production must therefore be reproducibly documented: source weights, quantization method, calibration data, and tool versions enter the genesis manifest so that every participant can retrace the derivation. Otherwise a trust anchor would arise at this point, which the protocol otherwise avoids everywhere.

In the reference implementation this reproducibility has been established and measured. The result of calibration is a small, versionable **scale pack**: the per-channel power-of-two scales, the lookup tables of the non-linear functions, and the hash of the source weights, together 1.8 megabytes for two models. From it every participant builds the integer artifact themselves, in under a minute rather than twenty, and compares its digest against the anchored one. That is the difference between a distributed artifact one has to trust and a recipe one can follow. For the genesis block it follows that the need for trust contracts to the *choice* of base model and calibration data: someone makes that choice, whereas the artifact itself can be rebuilt by anyone and checked against the anchored digest.

10.2 Model Updates

Updates (new versions θ_v+1) pass through a three-stage process:

1. **Proposal:** Treasury-funded or external teams submit candidate weights with a reproducible training/fine-tuning protocol.
2. **Shadow phase:** 5% of pod capacity runs the candidate in parallel; a public benchmark suite runs with on-chain attestation.
3. **Vote:** Voting weight = stake × work history (as in the validator election). On acceptance: a coordinated weight rollout over one transition epoch with dual version hosting.

In the long term, a second work class (training segments, verified à la Gensyn) can bring fine-tuning itself into the network; for v1 this is deliberately excluded (complexity).

10.3 Parameter Governance

Changeable by vote: sampling rate p, subsidy rate s, kernel whitelist, utilization target, dispute window. Not changeable (constitutional rank): total supply, the burn-and-mint principle, the determinism obligation of the runtime.

11. Open Research Questions

The following points are deliberately formulated as *measurement questions*: each names the quantity to be determined and the milestone in which this happens.

1. **Output quality of integer inference at the target scale (M0).** *Measured at two sizes (Ch. 6.9): a 2.11 percent gap at 0.5 and 1.14 percent at 7 billion parameters.* What remains open is the transfer to the intended scale, for which a more favorable result is to be expected but is not established. Two points are not a curve, and the only measurement that ever appeared to speak against the robustness assumption was an implementation error. The verification of Chapter 6 presupposes a model executable entirely in integer arithmetic. Eight-bit quantization is broadly established for transformers [18][19]; its extension to large language models is more recent [20] and not comprehensively validated at the intended scale. To be measured: the quality gap to the floating-point reference on established benchmarks. Should it prove too large, the basis of the chapter requires reassessment; the fallback would be tolerance-based commitments (point 10).
2. **Completeness of the execution specification (M0).** *Partial result available (Ch. 6.9):* the specification must be obtained from the weights, not from the documentation; on a reference implementation without accelerators it is

complete, including across separate, network-coupled processes. What remains open is the demonstration across different hardware classes and across physically separate hosts. The determinism property rests on all platform-dependent operations being covered. To be examined in particular: the behavior of integer matrix units at range boundaries, saturation semantics, possible compiler transformations, and the approximations of the non-linear functions. The result is a conformance suite with test vectors that admits new hardware without protocol changes.

3. **Throughput on heterogeneous hardware (M0/M1).** For integer inference, speedups over fp32 of 2.4 to 4.0 [18] and 3.7 to 4.1 [19] are reported. To be measured: whether this advantage appears uniformly across NVIDIA, AMD, Apple, and CPU hardware; uneven distribution would be a centralization risk.

4. **Grid width of token selection (M1).** Selection proceeds deterministically from quantized logits. To be measured: the quality impact of quantization and the frequency of boundary cases exactly on grid lines.

5. **Indistinguishability of control segments (M2).** The security gain of 6.7 stands or falls with miners being unable to recognize canaries. To be examined: by what statistical features they could be identified (prompt distribution, length, context construction, recurrence, timing), and whether admitting audited genuine segments removes those features. The share γ is further to be determined as a trade-off between deterrence and overhead.

6. **Pod latency versus local collusion density (M1).** The rule of Chapter 4.4 is qualitatively justified but unquantified. To be measured: achievable pipeline latency per zone size against the resulting local capacity concentration β_local.

7. **Activation compression (M1).** Bandwidth is the bottleneck of WAN operation. Integer activations favor compression, since they are already quantized. To be examined: whether delta encoding between successive tokens substantially reduces transmission volume. Condition: any compression must be protocol-defined and identical for both redundant pods (Ch. 6.5).

8. **Distributed prefix cache (M1/M2).** Shared prefixes promise considerable throughput gains. Before adoption, two security questions must be settled: the timing side channel through which an attacker could detect the existence of others' prefixes, and the condition that only Tier-1-confirmed prefixes may be cacheable.

9. **Integer training for language models (M3).** *A first partial result is available (Ch. 7.2):* over 200 steps on a model of 0.5 billion parameters the scheme holds with stochastic rounding (+0.67 percent on held-out text) and fails with round-to-nearest (+29.9 percent). What remains to be measured is the quality gap over a real training length and at the target scale, plus the gap between local loss blocks and global backpropagation. The literature evidence still comes from the image domain with comparatively small networks [23][24].

10. **Combining integer training with model growth (M3/M4).** *The sub-question on expansion is answered (Ch. 7.5):* it remains exactly function-preserving under integer representation, measured deviation 0.00e+00, provided the outgoing column is split rather than halved. What remains open is the combination in operation, that is whether a growth step in the middle of a running integer training reaches the same quality as in the literature on floating-point runs.

11. **Progressive growth under open network conditions (M4).** All literature stems from centrally controlled runs. To be investigated: behavior under heterogeneous capacity, interrupted runs, and VRF-assigned data.

12. **Financing of training.** The design finances training from treasury and an optional fee surcharge and states in Chapter 7.6 that every variant produces misaligned incentives. What is sought is a mechanism that compensates training contributions by benefit without introducing subjective assessment. This is the weakest point of the design.

13. **Attestation of external tools (M4).** For non-reproducible tool answers a trust anchor exists at the retrieving gateway (Ch. 8.1). To be investigated: for which classes of answer multiple retrieval by independent gateways is viable, and at what rate of change it fails.

14. **Residual effect of injected instructions (M4).** Architectural separation [39][40] prevents boundary violations but not poor decisions within the boundaries. To be measured: what damage is actually achievable through deception within a given budget.

15. **Model weights as a commons.** Unchanged and open: the weights necessarily reside with the miners, the model must be open-weight. Where does the base model come from? Added to this is the question of who performs and warrants the integer quantization, since the quantized model is part of θ_v (Ch. 10.2).

16. **Verification without a determinism requirement.** Appendix B.5 sets out why the tolerance schemes examined do not hold, in particular because of adaptive attackability. A scheme robust against an attacker who knows the verification criterion would remove the binding to quantized models. This is the most promising direction for a future edition.

17. **Confidentiality beyond class B.** Class C remains out of reach as long as computation occurs on plaintext (Ch. 9). TEE capacity as an optional class is designable but introduces vendor trust; homomorphic schemes remain a matter for observation.

18. **Isomorphic role assignment as an alternative to redundancy.** VeriLLM runs inference and verification roles on the same nodes, avoiding a separate pool of checkers [43]. To be investigated: whether this pattern is compatible with the lottery assignment of Chapter 4.3 without violating the independence assumption, and whether it could reduce the redundancy overhead below the 50 percent of point 19.

19. **Economics of the redundancy factor.** r = 2 halves efficiency versus centralized providers. It remains to be examined whether adaptive redundancy (r = 1 for miners with a long clean history) lowers cost without violating the independence assumption.

Appendix A: Core Data Types and Reference Algorithms

This appendix documents the protocol-relevant data types and the core algorithms of the reference implementation. Engineering documentation (repository structure, build conventions, implementation milestones, CI) can be found in the project repository under `docs/`.

All five sections of this appendix now have a tested implementation: `myl-types` (Merkle tree, VRF per RFC 9381, BLS12-381 signatures, the structs from A.1 with exact field order as a consensus contract), `myl-scheduler` (the five steps of A.2 as separate modules: hardware filter, geo-clustering, Fisher-Yates assignment, redundancy, sampling lottery), `myl-pod` (the mining loop from A.3, including tamper detection), `myl-verifier` (bisection, adjudication, and slashing from A.4), and `myl-ledger` (the state transitions from A.5, whose determinism is verified across independent runs). The blocks here are reference algorithms and stay terser than the code; adjudication, for instance, additionally binds the revealed activation to the hash committed in the trace, without which the comparison would be tautological. The current implementation status is documented in the repository, not in this paper.

A.1 Core Data Types (`myl-types`)

```
pub struct Segment {
    pub id: SegmentId,                      // h(session ‖ index)
    pub input_commitment: Hash,             // h(prompt_chunk ‖ kv_root)
    pub model_version: MerkleRoot,          // weights root θ_v incl. execution spec
    pub pod_path: Vec<MinerId>,             // pipeline order
    pub output_commitment: Hash,
    pub trace: Vec<ActivationHash>,         // h(a_0), …, h(a_k): computation trace
    pub signatures: Vec<BlsSignature>,      // one per shard transition
}

pub struct PoIBundle {
    pub epoch: EpochId,
    pub pod: PodId,
    pub segments_root: MerkleRoot,          // over all segment ids of the epoch
    pub vtfe_claimed: u64,                  // claimed work
    pub aggregate_sig: BlsSignature,        // aggregated over pod members
}

pub struct InferenceCredit { pub owner: Address, pub vtfe: u64, pub expiry: EpochId }
```

A.2 Epoch Scheduler (`myl-scheduler`), Deterministic Assignment

```
/// Reproducible identically by EVERY node, no central scheduler.
pub fn assign_epoch(
    seed: VrfOutput,                    // from the finalized block of the previous epoch
    miners: &[MinerRegistration],       // registration closes at epoch e−2
    latency_graph: &LatencyGraph,       // gossip-attested pairwise latencies
    cfg: &ShardConfig,                  // k shards, pod size k+2
) -> EpochAssignment {
    // 1. Filter miners by hardware class (VRAM ≥ shard size)
    // 2. Geo-clustering under latency constraint:
    //    form pods so that max pairwise latency in a pod < L_max (e.g. 80 ms),
    //    but cluster choice is seed-randomized (collusion protection, Ch. 9 item 2)
    // 3. Shard assignment WITHIN the pod: Fisher–Yates with seed
    // 4. Redundancy: every demand bucket → 2 disjoint pods
    // 5. Sampling lottery: mark p·|segments| segments for checkers
}
```

A.3 Pipeline Algorithm (`myl-pod`), the "Mining Loop"

Under uPoI, the mining loop is not hash guessing but the inference service loop:

```
/// Main loop of a shard miner (shard i in the pod)
async fn shard_loop(shard: ShardWeights, role: PodRole) {
    loop {
        // 1. Receive activations from the predecessor (or prompt embedding if i == 0)
        let (a_prev, seg) = recv_activations().await;

        // 2. Verify input hash against trace & set deterministic context
        verify_hash(&a_prev, seg.trace[i − 1])?;
        let ctx = DeterministicCtx::new(seg.id, seg.sampling_seed());

        // 3. Forward pass according to theta_v (integer; reduction order free, 6.2)
        let a_next = shard.forward_deterministic(&a_prev, &ctx);

        // 4. Extend the trace, sign, pass on
        let h_next = hash(&a_next);
        sign_transition(seg.id, seg.trace[i − 1], h_next);
        send_activations(next_peer(), a_next, seg).await;

        // 5. Update the session's KV cache locally (session affinity)
        kv_cache.update(seg.session_id, &a_next);

        // 6. Archive activations erasure-coded for the dispute window (DA duty)
        da_store.put(seg.id, i, encode_fragments(&a_prev));
    }
}

/// Pod coordinator: micro-batching + PoI aggregation
async fn coordinator_loop() {
    loop {
        let batch = intake.collect_microbatch(WINDOW_MS).await;  // pipelining
        dispatch_pipeline(batch).await;
        if epoch_boundary() {
            let bundle = build_poi_bundle(completed_segments());
            submit_to_consensus(bundle).await;                   // → myl-poi
        }
    }
}
```

A.4 Verification Protocol (`myl-verifier`)

```
/// Checker: recompute a sample; compare for bit equality.
async fn audit(seg: Segment) -> Option<Challenge> {
    let my_trace = rerun(&seg).await;                // own pass, same order
    let j = first_divergence(&seg.trace, &my_trace)?; // None ⇒ all correct
    Some(Challenge { seg_id: seg.id, layer_group: j, bond: CHECKER_BOND,
                     claimed_hash: my_trace[j] })
}

/// On-chain arbitration round (validator committee): exactly ONE shard forward.
/// The result is canonical: there is exactly one correct a_j.
fn adjudicate(ch: Challenge, fragments: DaFragments) -> Verdict {
    let a_in  = decode(fragments)?;                  // a_{j−1} from the DA layer
    let a_out = runtime::forward(ch.layer_group, &a_in);   // according to theta_v
    if hash(&a_out) == ch.claimed_hash { Verdict::SlashMiner }
    else                               { Verdict::SlashChecker }
}
```

A.5 Consensus Integration (`myl-consensus` + `myl-ledger`)

```
Block ::= { txs, poi_bundles, challenges, verdicts, epoch_meta }

State transitions (myl-ledger):
  burn_to_credits(addr, myl) → credit inference credits (myl / price_e)
  mint_amount(ema_burn)      → M_e from the EMA-smoothed burn
  distribute_mint(M_e)       → distribute rewards by the shares of 5.3
  apply_verdict(v)           → slash stake, pay bounty, reverse vTFE credit
  credit_spend(session, vtfe)→ debit session budget (agent contract)
```

Appendix B: Incentive Derivations

B.1 Security Condition for Shard Miners

A rational miner considers computing segments falsely (e.g., saving compute by returning random values). Model:

- `g` = gain per cheated segment (saved compute cost ≈ segment reward),
- `p` = probability that a segment is spot-checked,
- `S` = the miner's stake, fully slashed upon conviction,
- the miner cheats on a share `q` of its segments.

Redundancy (Tier 1) already detects fraud whenever the redundant pod is honest. Cheating thus pays only under collusion of both pods, or if the miner speculates that deviations are treated as errors without slashing. Conservatively, we consider the worst case where only sampling (Tier 2) sanctions fraud.

Expected gain per epoch with n segments: `E[G] = q·n·g`

Detection probability per epoch: `P_d = 1 − (1−p)^{q·n} ≈ q·n·p` (for small p·q·n)

Expected penalty: `E[S] = P_d · S ≈ q·n·p·S`

Honesty is dominant if `E[S] > E[G]` for all q > 0:

$$qnpS > qng \Leftrightarrow S > g/p$$

The sharper bound $S_{min} = g/p^2$ stated in the main text follows when one additionally requires that cheating does not amortize over the time horizon until the first expected audit (≈ 1/p segments), i.e., against miners planning a short cheating window followed by exit (hit-and-run):

$$S_{min} = g/p \cdot 1/p = g/p^2$$

where g/p corresponds to the gain until expected detection.

Numerical example: p = 0.02, segment reward g = 0.5 MYL yields S_{min} = 1250 MYL per segment of capacity. That is 2500 segment rewards. A miner with a capacity of 100 segments per epoch earns 50 MYL per epoch and therefore pledges roughly 25 epochs of income. With one-hour epochs: about one day's income as a pledge per unit of capacity, practicable, and tightenable via p if fraud cases are observed.

B.2 Collusion Probability of Redundancy

Let β be the share of colluding miner capacity. Both redundant pods (each with k shard positions) must collude fully to push a false segment through Tier 1:

$$P_{coll} \approx \beta^{2k}$$

At β = 0.2 and k = 8: P_coll ≈ $6.6 \cdot 10^{-12}$. Even at β = 0.5: $1.5 \cdot 10^{-5}$, and every such segment still carries the sampling risk p with full slashing of *all* 2k participating stakes. The geographic clustering of pod formation (Ch. 4.3) raises β locally; an analysis planned for milestone M1 (Appendix B.9) is to quantify how much seed randomness must be mixed into cluster selection to keep β_local below a target bound (Ch. 9 item 2).

B.3 Checker Incentives

Checker compensation = base pay (4% of minting, proportional to audited volume) + bounty `b·S` from slashes (b = 30%). The base pay ensures auditing remains profitable even at fraud rate ≈ 0, since the system must not depend on the existence of fraud. False challenges cost the bond; the bond is sized so that spam challenges (forcing costly arbitration rounds) are unprofitable: bond > cost of the on-chain arbitration round × safety factor.

B.4 Self-Dealing (Formalization of 5.6)

An attacker with capacity share α burns X MYL to harvest minting. Return: `α · M_e`. In equilibrium (M_e ≈ $\bar{B}$_e, EMA-damped), the marginal return of an additional burn ΔX satisfies:

$$\alpha \cdot \Delta X \cdot w < \Delta X \text{ for all } \alpha < 1/w$$

with EMA weight w < 1 and the attacker's capacity share α.

Since w ≈ 1/30 (EMA window) and α ≤ 1, self-dealing is strictly loss-making in equilibrium.

Subsidy phase (s > 0). Sharpened condition: The model calculation shows that in the bootstrap phase the pure burn-vs-mint comparison is not sufficient: with minting `M_e = B̄_e·(1+s)`, a self-dealer nominally harvests more than they burn. Security here rests on the work binding of minting. Rewards flow only against verified computational work, whose real costs (hardware, electricity; share c of the reward, empirically c ≈ 0.6–0.8) the attacker bears like any miner. Self-dealing is loss-making exactly if

$$s < c/(1 - c)$$

At c = 0.7, therefore s < 2.33, the starting subsidy s = 0.5 lies far below. This inequality is to be maintained as a **governance invariant**: s must never be raised anywhere near c/(1−c).

B.5 Integer Execution and the Rejected Alternatives

The verification of Chapter 6 rests on an arithmetic property and two additional stipulations. Both are substantiated below; the corresponding programs are indexed in B.9.

B.5.1 Associativity as the foundation. Simulating a reduction over 8,192 terms, corresponding to one matrix row, and comparing four orderings (sequential, pairwise tree, split-K with eight partial sums, random order), the integer computation yields identical results in all 200 runs. The same computation in single-precision floating point agrees across all orderings in only 9 of 200 cases; in 96 percent of cases the results diverge. The determinism of integer execution is therefore not a constraint on the implementation but a property of the operation.

B.5.2 Overflow margin. With int8 factors the largest product magnitude is 127 · 127 = 16,129. A 16-bit accumulator therefore carries only two terms and is ruled out. A 32-bit accumulator carries over 133,000 terms by calculation; the empirically largest sum over 8,192 terms was around 1.3 million, a margin of roughly a factor of 1,639. A 32-bit accumulator is thus adequately dimensioned. Behavior at the range boundary (saturation) must nonetheless be specified so that it does not remain implementation-dependent.

B.5.3 Non-linear operations and dynamic quantization. An integer Softmax approximation modeled on [18][20] yielded identical results under three different summation orderings in 100 of 100 cases. Dynamic quantization likewise proved order-independent: the scaling factor derived from the magnitude maximum agreed in 200 of 200 cases, since maximum formation and integer division do not depend on element order. The parts of inference beyond plain matrix multiplication therefore remain deterministic as well.

B.5.4 The one remaining pitfall: division of negative numbers. Flooring division, truncation toward zero, and arithmetic right shift do not agree for negative operands: −7 divided by 2 yields −4 or −3 depending on convention. In three of five cases examined the methods diverged. Since programming languages differ here, this would be a real source of platform-dependent results. Stipulating the arithmetic right shift resolves the problem entirely: across 100,000 random cases the shift agreed without exception with flooring division, and it is identically defined as an instruction on all common architectures. Unlike an ordering prescription, this stipulation costs no throughput and is fully testable with finitely many test vectors.

B.5.5 Why floating point with fixed ordering was rejected. The approach is proven [15] but does not hold for the present case. It restricts parallelization and thus costs throughput; it presupposes uniform rounding behavior of the individual instruction, which is not the case for matrix units designed for AI [21]; and it has so far been worked out only for the single-device case, while reproducibility across multiple nodes with pipeline parallelism is named as future work [15]. A centralizing effect compounds this: a mandate favoring high-precision accumulation disadvantages consumer accelerators, on which that path frequently runs at only half rate, while data-center hardware knows no such penalty.

B.5.6 Why a tolerance model was rejected. A distance comparison below a threshold τ [14] was examined in four respects. First, an admissible τ requires, even under favorable assumptions, that manipulations produce at least five times the legitimate noise; under violated distributional assumptions the requirement rises to twenty or thirty-five times. Second, noise accumulates across chained execution so strongly that after a few layers the results of two honest nodes diverge as much as manipulated ones do from unmanipulated. Third, a structure-based criterion does not detect precision manipulations, since quantization barely alters the ranking of dominant components. Fourth, and decisively: a tolerance band is adaptively attackable. Whoever knows the criterion computes the audited components correctly and falsifies the rest; in simulation such an attack remained undetected across ten subsequent layers while saving a substantial share of the computational work.

B.5.7 Limits of these analyses. These are model calculations in software, not measurements on accelerators. What is established is the arithmetic property, not the bit equality of a complete transformer inference on real hardware. Open questions remain in particular regarding the behavior of integer matrix units at range boundaries, possible compiler transformations, and whether the execution specification is complete. Unlike with floating point, however, these cases are enumerable and testable through conformance suites (Ch. 10, point 2).

B.6 Evidence on Training

The statements of Chapter 7 rest on model calculations whose programs are indexed in B.9.

B.6.1 Determinism of the backward pass. Gradient computation is a sum of products of activation and error term and thus as associative as the forward pass. Across 200 runs with three summation orderings, results were identical without exception. The verification model of Chapter 6 therefore carries over unchanged.

B.6.2 Overflow and block scaling. Without countermeasures, error terms with eight-bit weights and a 32-bit accumulator exceed the range after two backward steps and then grow exponentially to 78 bits. With the block scaling of [23], the error vector remains stable at roughly fifteen bits over forty layers; no overflow occurs. Since the scaling exponent follows from the magnitude maximum and is applied as an arithmetic right shift, the operation remains order-independent.

B.6.3 Training capacity. For a model of the 24-billion class, an effort of $6 \cdot N$ FLOPs per token, and redundancy factor 2, a base rate of ten percent yields: 500 miners reach roughly 98 million tokens daily, 5,000 miners roughly one billion, 50,000 miners roughly nine billion. A fine-tuning run of one billion tokens is thus achievable in a day from medium network size onward. Pre-training, requiring trillions of tokens, remains out of reach by orders of magnitude.

B.6.4 Cost of data provenance. For a corpus of one billion documents the Merkle depth is 30, so a single proof is 960 bytes against 8,192 bytes of payload per segment, that is 11.7 percent overhead. If contiguous segments are assigned in batches, they share the common subtree, and they share all of it: whoever holds every leaf of a subtree computes its root from those leaves and needs no sibling node at all for the lower levels. Only the path from the subtree root to the tree root is transmitted, that is $30 - \log_2 n$ nodes for the whole batch rather than per segment. At 16 segments this comes to 832 bytes against 128 kilobytes of payload, or 0.63 percent; at 256 segments to 704 bytes against 2 megabytes, or 0.03 percent. This assumes a batch aligned to the grid; an arbitrary contiguous range decomposes into a few complete subtrees and stays in the same order of magnitude.

B.6.5 Selection poisoning. Under free choice of data, an attacker's influence equals their capacity share: forty percent share yields forty percent influence over data composition. Under VRF assignment, only refusal of assigned segments remains; residual influence falls to roughly two percent and additionally becomes visible through the refusal rate.

B.6.6 Robust aggregation. Simulated was the deviation of the aggregated gradient from the honest value under Byzantine contributions. The mean deviates by 0.76 at a five percent attacker share and by 3.80 at twenty percent. The median stays at 0.03 and 0.10 respectively and holds even at a one-third attacker share (0.19). The trimmed mean fails there as expected (3.91), since twenty percent trimming does not suffice. Hence the choice of the median, whose breakdown point lies at fifty percent.

B.6.7 Stale gradients. On a convex objective the asynchronous procedure converges even at up to fifty steps of delay. This statement is qualified: the model is quadratic and thus considerably more benign than the loss landscape of a language model. What is established is only that delay poses no fundamental obstacle; the practical limit remains to be measured.

B.6.8 Benchmark manipulation. A test set known in advance permits, in the model calculation, roughly 35 percent apparent progress without genuine improvement. If the hold-out set is drawn by VRF from the corpus only after training concludes, optimization toward it is precluded, since the selection is neither predictable nor retroactively influenceable.

B.6.9 Catastrophic forgetting. Without replay data, performance on existing capabilities falls to roughly forty percent in the model calculation. A five percent replay share raises it to sixty percent, fifteen percent to eighty-two percent, with correspondingly lower gains on the new data. The replay share is therefore a trade-off parameter, not an optimum.

B.6.10 Model growth. A growth step from 24 to 32 billion parameters requires roughly 200 billion tokens against 640 billion for a pre-training run of the same size, a ratio of about 1 to 3.2; later steps lie at about 1 to 3.0. At a ten percent base rate the first step takes over seven years with 500 miners, roughly 263 days with 5,000 miners, and roughly thirty days with 50,000 miners. Depth grows approximately with the square root of parameter count, so a model growing from 24 to 100 billion parameters grows from roughly 48 to 98 layers and thus from five to ten shards.

B.6.11 Word width of the integer master. The width follows from the smallest change that must still arrive. Measured on a model of 0.5 billion parameters at a learning rate of 1e-5, a single update step amounts to a median of 6.4e-06 of an int8 grid level, and 5.7e-07 at the first percentile. So that such a step does not round to zero, the master needs `F` fractional bits below the eight-bit level with `2^−F` beneath the step size: eighteen bits suffice at the median, twenty-one at the first percentile. Twenty-five are recommended, that is four bits of reserve for smaller learning rates later in a run; the master is then 33 bits wide and fits into a 64-bit word. A second bound applies to aggregation: summing ten million contributions in units of the least significant master bit reaches about 2.15e09 and thus already just exceeds a signed 32-bit accumulator (2.147e09). The aggregation of 7.4 is therefore to be carried in 64 bits. These values stem from a measurement on the reference implementation, not from a model calculation.

B.6.12 Limits of these analyses. The values of sections B.6.1 through B.6.10 stem from model calculations in software, not from training runs; B.6.11, by contrast, is measured on the reference implementation. The capacity calculation rests on an efficiency assumption of 25 percent of peak performance over WAN; the growth calculation on a conservatively assumed fifty percent saving relative to pre-training. The models for forgetting and benchmark manipulation are qualitative and serve orders of magnitude, not prediction.

B.7 Training and the Burn-and-Mint Cycle

Training generates work but no burn. It was examined whether this disturbs the cycle of Chapter 5.

B.7.1 Effect on net inflation. Over 2,000 epochs, pure inference operation yields net inflation of 11.8 percent, carried by the expiring subsidy. Financing training from additional minting raises it to 23.0 percent, thus nearly doubling it. Financing from the treasury leaves it unchanged, since only existing minting is redistributed; a fee surcharge lowers it slightly to 11.76 percent, since it raises the burn to the same degree as the expenditure. Hence the stipulation in Chapter 5.3.

B.7.2 Reference quantities. The base rate of Chapter 7.1 is measured against free capacity, the treasury share of Chapter 5.3 against minting. At seventy percent utilization, ten percent of free capacity corresponds to roughly three percent of total output. Both quantities are thus compatible; the treasury fully covers training shares up to about three percent of minting.

B.7.3 Displacement of inference. Miners choose between the two work classes by compensation per compute hour. If training compensation lies below, training is performed only with free capacity, as intended. At parity, assignment alone decides; above it, training displaces inference and thus the network's source of revenue. The upper bound of Chapter 5.3 is therefore not a fine adjustment but a stability condition.

B.7.4 Feedback through demand. Simulated was the case in which training improves model quality and thereby raises demand. Even a weak effect measurably increases cumulative burn volume, a marked effect substantially. Training thus finances itself over the long run through the cycle, provided the quality improvement actually materializes. If it does not, the expenditure is a pure loss borne by the treasury, where it remains visible.

B.7.5 Limits. As with the other calculations in this appendix, this is a model, not a measurement. The feedback between model quality and demand in particular is set qualitatively; its actual strength is the decisive open quantity for the economics of training (Ch. 11, point 12).

B.8 Issuance Structure

Evidence for Chapter 5.7.

B.8.1 Launch-phase requirement. At the target rate of two percent, S_{min} per capacity unit is 1,250 MYL. For fifty initial miners this yields a stake requirement of 62,500 MYL, against a credit requirement of the first users of roughly 540 MYL. The stake therefore alone determines the order of magnitude of the initial quantity.

B.8.2 Effect of the sampling rate. Since S_{min} depends quadratically on p, the requirement for two hundred miners falls from 250,000 MYL at two percent to 40,000 at five, 10,000 at ten, 1,600 at twenty-five, and 400 MYL at fifty percent. A raised audit rate during the launch phase thus reduces the necessary initial quantity by more than two orders of magnitude.

B.8.3 Effect of an emission cap. Ten years were simulated under growing demand. Without a cap, circulation rises from 100,000 to roughly 31 million MYL, with minting following the smoothed burn. A cap above initial minting has no effect at first but binds as demand grows, letting circulation fall back to 150,000 MYL after ten years. A cap binding from the outset holds circulation permanently at about 100,000 MYL. In both cases more is burned than minted, so work performed is no longer fully compensated. A cap therefore acts not as a guarantee of scarcity but as a brake on capacity.

B.8.4 Early-phase concentration. With annual minting halving, roughly 28 percent of five-year emission falls in the first year, regardless of whether the network grows to 500, 5,000, or 50,000 miners. The advantage of early participation thus does not depend on later growth but solely on the course of the subsidy curve.

B.8.5 Limits. Demand development is modeled as steady growth with log-normal fluctuation. Abrupt demand changes, network forks, and external price effects are not represented.

B.9 Index of Simulations

The model calculations of this appendix are included as executable programs in the project repository under `README/Whitepaper/simulations/`. They require no dependencies beyond the standard library.

Program	Subject	Referenced in
`tokenomics_sim.py`	Burn-and-mint equilibrium, self-dealing across the subsidy phase	B.4
`tau_sim.py`	Required separability of a tolerance scheme	B.5.1
`robustness_sim.py`	Sensitivity of that separability to violated distributional assumptions	B.5.2
`hardware_noise_sim.py`	Noise levels of common accelerator classes from specification data	B.5.3
`accum_alternatives_sim.py`	Intermediate solutions for accumulation precision	B.5.3
`topk_stability_sim.py`	Structure-based commitments and the adaptive attack	B.5.4
`integer_determinism_sim.py`	Associativity, overflow margin, division semantics	B.5.1, B.5.4
`integer_training_sim.py`	Determinism of the backward pass, block scaling	B.6.1, B.6.2
`training_capacity_sim.py`	Training throughput, cost of data provenance, selection poisoning	B.6.3 to B.6.5
`training_integrity_sim.py`	Robust aggregation, stale gradients, benchmark manipulation, forgetting	B.6.6 to B.6.9
`model_growth_sim.py`	Cost and timescale of growth steps	B.6.10
`training_tokenomics_sim.py`	Interaction of training with the burn-and-mint cycle	B.7
`genesis_supply_sim.py`	Launch phase, emission trajectory, early-phase concentration	B.8
`latency_sim.py`	Pod latency versus local collusion density (planned, milestone M1)	B.2

References

1. Jimenez et al.: HadAgent: Harness-Aware Decentralized Agentic AI Serving with Proof-of-Inference Blockchain Consensus. arXiv:2604.18614, 2026. https://doi.org/10.48550/arXiv.2604.18614

2. Qubic Project: Useful Proof of Work / Aigarth. Project documentation. https://docs.qubic.org/

3. Rao et al.: Bittensor: A Peer-to-Peer Intelligence Market. Whitepaper, 2021. https://www.bittensor.com/whitepaper

4. Borzunov et al.: Petals: Collaborative Inference and Fine-tuning of Large Models. arXiv:2209.01188, 2022. https://doi.org/10.48550/arXiv.2209.01188

5. Gensyn: Litepaper: Verifiable Deep Learning Compute Protocol. Technical report. https://docs.gensyn.ai/litepaper

6. Conway et al.: opML: Optimistic Machine Learning on Blockchain. arXiv:2401.17555, 2024. https://doi.org/10.48550/arXiv.2401.17555

7. Design and Evaluation of Cost-Aware Proof of Quality for Decentralized LLM Inference. arXiv:2512.16317, 2025. https://doi.org/10.48550/arXiv.2512.16317

8. PolyLink: A Blockchain-Based Decentralized Edge AI Platform for LLM Inference. arXiv:2510.02395, 2025. https://doi.org/10.48550/arXiv.2510.02395

9. DeServe: Towards Affordable Offline LLM Inference via Decentralization. arXiv:2501.14784, 2025. https://doi.org/10.48550/arXiv.2501.14784

10. Teutsch, Reitwießner: A Scalable Verification Solution for Blockchains (Truebit). Whitepaper 2017, arXiv:1908.04756. https://doi.org/10.48550/arXiv.1908.04756

11. Kalodner et al.: Arbitrum: Scalable, Private Smart Contracts. USENIX Security, 2018. https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner

12. Yin et al.: HotStuff: BFT Consensus in the Lens of Blockchain. PODC, 2019. https://doi.org/10.1145/3293611.3331591

13. Nakamoto: Bitcoin: A Peer-to-Peer Electronic Cash System. Whitepaper, 2008. https://bitcoin.org/bitcoin.pdf

14. Ong et al.: TOPLOC: A Locality-Sensitive Hashing Scheme for Trustless Verifiable Inference. arXiv:2501.16007, 2025. https://doi.org/10.48550/arXiv.2501.16007

15. Arun et al.: Verde: Verification via Refereed Delegation for Machine Learning Programs. arXiv:2502.19405, 2025. https://doi.org/10.48550/arXiv.2502.19405

16. Microsoft Research: RepDL: Reproducible Deep Learning. Software library. https://github.com/microsoft/RepDL

17. Dettmers et al.: LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339, 2022. https://doi.org/10.48550/arXiv.2208.07339

18. Kim et al.: I-BERT: Integer-only BERT Quantization. ICML, 2021. arXiv:2101.01321. https://doi.org/10.48550/arXiv.2101.01321

19. Li, Gu: I-ViT: Integer-only Quantization for Efficient Vision Transformer Inference. arXiv:2207.01405, 2022. https://doi.org/10.48550/arXiv.2207.01405

20. Hu et al.: I-LLM: Efficient Integer-Only Inference for Fully-Quantized Low-Bit Large Language Models. arXiv:2405.17849, 2024. https://doi.org/10.48550/arXiv.2405.17849

21. Khattak, Mikaitis: Accurate Models of NVIDIA Tensor Cores. arXiv:2512.07004, 2025. https://doi.org/10.48550/arXiv.2512.07004

22. Song et al.: PocketNN: Integer-only Training and Inference of Neural Networks via Direct Feedback Alignment. arXiv:2201.02863, 2022. https://doi.org/10.48550/arXiv.2201.02863

23. Wang et al.: NITI: Training Integer Neural Networks Using Integer-only Arithmetic. IEEE TPDS, 2022. arXiv:2009.13108. https://doi.org/10.48550/arXiv.2009.13108

24. Pirillo et al.: NITRO-D: Native Integer-only Training of Deep Convolutional Neural Networks. arXiv:2407.11698, 2024. https://doi.org/10.48550/arXiv.2407.11698

25. Chen et al.: Net2Net: Accelerating Learning via Knowledge Transfer. arXiv:1511.05641, 2015. https://doi.org/10.48550/arXiv.1511.05641

26. Chen et al.: bert2BERT: Towards Reusable Pretrained Language Models. ACL, 2022. https://aclanthology.org/2022.acl-long.151/

27. Gong et al.: Efficient Training of BERT by Progressively Stacking. ICML, 2019, PMLR 97, pp. 2337-2346. https://proceedings.mlr.press/v97/gong19a.html

28. Wang et al.: Learning to Grow Pretrained Models for Efficient Transformer Training (LiGO). ICLR, 2023. arXiv:2303.00980. https://doi.org/10.48550/arXiv.2303.00980

29. Du et al.: Stacking Your Transformers: A Closer Look at Model Growth for Efficient LLM Pre-Training. NeurIPS, 2024. arXiv:2405.15319. https://doi.org/10.48550/arXiv.2405.15319

30. Blockchain-enabled Data Integrity for Federated Learning: Merkle-based Provenance and Auditable Update Trails. Discover Artificial Intelligence, 2026. https://link.springer.com/journal/44163

31. Yang et al.: TrustDFL: A Blockchain-Based Verifiable and Trusty Decentralized Federated Learning Framework. Electronics 13(1), Art. 86, 2024. https://doi.org/10.3390/electronics13010086

32. PoCQ: Proof of Contribution Quality as a Lightweight Blockchain Consensus for Secure Federated Learning. arXiv:2606.05642, 2026. https://doi.org/10.48550/arXiv.2606.05642

33. FIDELIS: Blockchain-Enabled Protection Against Poisoning Attacks in Federated Learning. arXiv:2508.10042, 2025. https://doi.org/10.48550/arXiv.2508.10042

34. Blanchard et al.: Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent (Krum). NeurIPS, 2017, pp. 119-129. https://proceedings.neurips.cc/paper/2017/hash/f4b9ec30ad9f68f89b29639786cb62ef-Abstract.html

35. Yin et al.: Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates. ICML, 2018, PMLR 80, pp. 5650-5659. https://proceedings.mlr.press/v80/yin18a.html

36. Bentov et al.: Cryptocurrencies without Proof of Work. arXiv:1406.5694, 2014. https://doi.org/10.48550/arXiv.1406.5694

37. KRNC: New Foundations for Permissionless Byzantine Consensus and Global Monetary Stability. arXiv:1909.07433, 2019. https://doi.org/10.48550/arXiv.1909.07433

38. Delgado Fernandez et al.: Agent-based Model of Initial Token Allocations: Evaluating Wealth Concentration in Fair Launches. arXiv:2208.10271, 2022. https://doi.org/10.48550/arXiv.2208.10271

39. Willison: The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. Blog post, 2023. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/

40. Debenedetti et al.: Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025. https://doi.org/10.48550/arXiv.2503.18813

41. Lin et al.: Towards Fully 8-bit Integer Inference for the Transformer Model. IJCAI, 2020. arXiv:2009.08034. https://doi.org/10.48550/arXiv.2009.08034

42. Jacob et al.: Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR, 2018. arXiv:1712.05877. https://doi.org/10.48550/arXiv.1712.05877

43. VeriLLM: A Lightweight Framework for Publicly Verifiable Decentralized Inference. arXiv:2509.24257, 2026. https://doi.org/10.48550/arXiv.2509.24257

44. Frantar et al.: GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR, 2023. arXiv:2210.17323. https://doi.org/10.48550/arXiv.2210.17323

45. Gupta et al.: Deep Learning with Limited Numerical Precision. ICML, 2015. arXiv:1502.02551. https://doi.org/10.48550/arXiv.1502.02551

46. Seide et al.: 1-Bit Stochastic Gradient Descent and its Application to Data-Parallel Distributed Training of Speech DNNs. Interspeech, 2014, pp. 1058-1062. https://www.isca-archive.org/interspeech_2014/seide14_interspeech.html

47. Karimireddy et al.: Error Feedback Fixes SignSGD and other Gradient Compression Schemes. ICML, 2019. arXiv:1901.09847. https://doi.org/10.48550/arXiv.1901.09847

Note on Scholarly Attribution

Reference [1] anticipates the term and core idea of proof of inference; references [14] and [15] contain the two verification building blocks that the model of Chapter 6 combines. The delineation of this work's own contributions is set out in Chapter 2. Where a DOI exists it is given; for project documentation, whitepapers, and works without an assigned DOI, the stable source reference is provided instead.

Declaration on the Use of AI Tools

This work was produced with the use of AI-assisted systems (Claude Fable, Opus, and Sonnet, Anthropic; Qwen3.8-Max, Alibaba). The system was used for the linguistic elaboration of the text from the author's specifications, for literature research, for the formalization of the derivations in Appendix B, and for reference code, simulation, and typesetting. Concept, architectural decisions, and protocol parameters originate with the author. The results were reviewed by the author but not independently replicated throughout. The source code is a reference implementation without production readiness and without external security audit; the simulations rest on model assumptions, not on operational data. The author bears full responsibility for the content.

This is the English edition of the whitepaper. The German original ("Myelith, Ein dezentrales Netzwerk, in dem Konsensarbeit ein agentisches Sprachmodell betreibt") is published alongside it; in case of discrepancies, the German original prevails.